西安交通大学

操作系统专题实验报告

班级：_____

学号：_____

姓名：_____

2025年 12月 2日

目 录

1 openEuler 系统环境实验.....4
1.1 实验目的.....4
1.2 实验内容.....4
1.3 实验思想.....5
1.4 实验步骤.....5
1.5 进程相关编程实验.....6
1.6 线程相关编程实验.....9
1.7 自旋锁实验.....10
1.8 实验总结.....13
1.8.1 实验中的问题与解决过程.....13
1.8.2 实验收获.....13
1.8.3 意见与建议.....16
1.9 附件.....16
1.9.1 附件 1 程序.....16
1.9.2 附件 2 Readme.....20
2 进程通信与内存管理.....29
2.1 实验目的.....29
2.2 实验内容.....29
2.3 实验思想.....30
2.4 实验步骤.....30
2.5 进程的软中断通信.....31
2.6 进程的管道通信.....36
2.7 页面的置换.....40
2.8 实验总结.....44
2.9.1 实验中的问题与解决过程.....44
2.9.2 实验收获.....45
2.9.3 意见与建议.....46
2.9 附件.....47
2.10.1 附件 1 程序.....47
2.10.2 附件 2 Readme.....54
3 文件系统.....71
3.1 实验目的.....71
3.2 实验内容.....71
3.3 实验思想.....71

3.4 实验步骤 72
3.5 动态模块加载与系统调用篡改 73
3.6 Linux 设备驱动 79
3.7 实验总结 90
3.6.1 实验中的问题与解决过程 90
3.6.2 实验收获 91
3.6.3 意见与建议 92
3.8 附件 93
3.7.1 附件 1 程序 93
3.7.2 附件 2 Readme 104

1 openEuler 系统环境实验

1.1 实验目的

（1）熟悉华为云服务器上 openEuler 操作系统基本系统环境；

（2）理解进程的创建、进程地址空间的概念、父子进程资源的关系；

（3）观察进程调度，了解进程调度的过程，了解孤儿进程和僵尸进程的区别；

（4）理解线程与进程的关系，对等线程间的关系。

（5）理解并运用信号量及其 PV 操作、自旋锁实现线程间的同步互斥。

1.2 实验内容

（1）在华为云上搭建 openEuler 操作系统环境，通过运行 shell 命令查看系统信息以了解并熟悉 openEuler 操作系统。

（2）熟悉操作命令、编辑、编译、运行程序。完成给定程序（图 1.1 所示的程序）的运行验证，多运行程序几次观察结果；去除 wait 后再观察结果并进行理论分析。

（2）在所给程序中添加一个全局变量，在父进程和子进程中分别对这个变量做不同操作并输出操作结果，对结果进行观察并解释，同时输出两种变量的地址观察并分析。

（3）修改程序，在子进程中调用 system 函数和在子进程中调用 exec 族函数以执行自己写的一段程序，在此程序中输出进程 PID，进行比较分析。

（4）线程实验：在进程中给一变量赋初值并创建两个线程，在两个线程中分别对此变量循环 5000 次以上做不同的操作并输出结果；多运行几遍程序观察运行结果，如果发现每次运行结果不同，请解释原因并修改程序解决，考虑如何控制互斥和同步；在线程中调用 system 函数/exec 族函数以执行自己写的一段程序，输出进程 PID 和线程 TID, 进行比较分析。

（5）自旋锁实验：在进程中给一变量赋初值并成功创建两个线程；在两个线程中分别对此变量循环 5000 次以上做不同的操作并输出结果；使用自旋锁实现互斥和同步。

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid, pid1;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
      fprintf(stderr, "Fork Failed");
      return 1;
    }
    else if (pid == 0) { /* child process */
      pid1 = getpid();
      printf("child: pid = %d",pid); /* A */
      printf("child: pid1 = %d",pid1); /* B */
    }
    else { /* parent process */
      pid1 = getpid();
      printf("parent: pid = %d",pid); /* C */
      printf("parent: pid1 = %d",pid1); /* D */
      wait(NULL);
    }

    return 0;
}
```

图 1.1 给定程序

1.3 实验思想

openEuler 基于 Linux 内核，支持多种架构，支持多种虚拟化技术，支持多种容器技术，支持多种云计算技术。openEuler 采用 Linux 内核，通过了解 openEuler 系统下的 shell 命令和系统命令后，所学习的知识可以轻松迁移到其他 Linux 发行版本中，如 Ubuntu。

通过配置华为云服务器上 openEuler 操作系统基本系统环境，运行 shell 命令查看系统信息以达到了解和使用 openEuler 操作系统的目的。

通过进程相关编程实验，编写和运行简单的进程、线程相关程序，理解进程与线程概念、进程空间与物理内存空间、进程调度、进程间变量管理、进程调用其他程序、如何实现正确的并发（同步互斥）等方面在操作系统中的实际操作。具体来讲，通过输出子进程和父进程的 PID，观察进程调度，了解进程调度的过程，了解孤儿进程和僵尸进程的区别。观察并发进程中的全局变量改变，输出父子进程共享变量地址以了解物理地址与虚地址概念，从而理解进程地址空间的概念，了解关于地址绑定的基础知识。创建两个线程运行后体会线程与进程的关系、对等线程间的资源共享以及同步与互斥的知识。

1.4 实验步骤

（1）登录华为云,搭建 openEuler 操作系统环境。

（2）编辑、编译、运行给定的程序，多次运行观察实验结果。

（3）去除 wait()后观察结果并进行理论分析, 比较程序在有无 wait()函数时的运行结果，分析 wait()函数的作用。

（4）添加一个全局变量，在父进程和子进程中对这个变量做不同操作，输出操作结果，同时输出两种变量的地址并进行分析。

（5）在子进程中调用 system 函数执行自己写的一段程序，在此程序中输出进程 PID 和其父进程 PID 进行比较分析。

（6）在子进程中调用 exec 函数执行自己写的一段程序，在此程序中输出进程 PID 和其父进程 PID 进行比较分析。

（7）修改给定程序，给一变量赋初值并创建两个线程，在两个线程中分别对此变量循环 5000 次做不同的操作并输出结果。

（8）多运行几遍程序观察运行结果。

（9）完成上述过程的同步和互斥操作。

（10）在两个线程中调用 system 函数执行自己写的一段程序，在此程序中输出线程 TID 及进程 PID,进行分析。

（11）在两个线程中调用 exec 函数执行自己写的一段程序，在此程序中输出线程 TID 及进程 PID,进行分析。

（12）参考实验指导书，编写模拟自旋锁程序代码，补充主函数代码，用自旋锁实现线程间的同步。编译并运行程序，分析运行结果。

1.5 进程相关编程实验

步骤一：编写并多次运行图 1-1 中代码，运行结果：

```
[root@kp-test01 test]# ./test1
parent: pid = 5089
child: pid = 0
parent: pid1 = 5088
child: pid1 = 5089
[root@kp-test01 test]# ./test1
child: pid = 0
parent: pid = 5091
child: pid1 = 5091
parent: pid1 = 5090
[root@kp-test01 test]# ./test1
parent: pid = 5093
child: pid = 0
parent: pid1 = 5092
```

分析：fork()的作用：创建一个与父进程几乎完全相同的子进程（pid 与地址空

间不同），其返回值：在子进程中为 0，在父进程中为子进程的 pid 值，若创建失败则返回值<0。

fork()后父子进程拥有独立的执行流，它们并行运行，操作系统的调度策略决定了输出顺序的不确定性。

步骤二：删去图 1-1 代码中的 wait()函数并多次运行程序，运行结果：

```
[root@kp-test01 test]# gcc -o test1 test1.c
[root@kp-test01 test]# ./test1
child: pid = 0
parent: pid = 5100
child: pid1 = 5100
parent: pid1 = 5099
[root@kp-test01 test]# ./test1
parent: pid = 5102
child: pid = 0
parent: pid1 = 5101
child: pid1 = 5102
[root@kp-test01 test]# ./test1
parent: pid = 5105
child: pid = 0
parent: pid1 = 5104
child: pid1 = 5105
```

分析：理论分析上，删除 wait() 后，父进程可以不再等待子进程，则可能先于子进程结束，导致子进程变为孤儿进程；若父进程未正确处理子进程退出，可能会产生“僵尸进程”。但是此处输出并未体现这些现象。

步骤三：修改图 1-1 中代码，增加一个全局变量并在父子进程中对其进行不同的操作，观察并解释所做操作和输出结果。

```
[root@kp-test01 test]# gcc -o test1 test1.c
[root@kp-test01 test]# ./test1
parent: pid = 5136
child: pid = 0
parent: pid1 = 5135
child: pid1 = 5136
parent: common = 161, addr = ffffc7107004
child: common = 163, addr = ffffc7107004
[root@kp-test01 test]# ./test1
parent: pid = 5138
child: pid = 0
parent: pid1 = 5137
child: pid1 = 5138
parent: common = 161, addr = ffffc325ede4
child: common = 163, addr = ffffc325ede4
```

操作：增设全局变量 common = 162，在父进程中++common，子进程中—common，并输出 common 的地址。

分析：两进程对 common 分别进行了进行了加减操作，并未产生干扰，说明父子进程在 fork()后各自拥有独立的虚拟地址空间。虽然变量名相同，但物理上存储在不同的内存页中，互不影响。二者地址相同，体现虚拟地址空间的独立性。

步骤四：在步骤三基础上，在 return 前增加对全局变量的操作并输出结果，观察并解释所做操作和输出结果。

```
[root@kp-test01 test]# gcc -o test1 test1.c
[root@kp-test01 test]# ./test1
parent: pid = 5210
child: pid = 0
parent: pid1 = 5209
child: pid1 = 5210
parent: common = 161, addr = ffffdaa04bf4
child: common = 163, addr = ffffdaa04bf4
common = 171, addr = ffffdaa04bf4
common = 173, addr = ffffdaa04bf4
[root@kp-test01 test]# ./test1
parent: pid = 5212
child: pid = 0
parent: pid1 = 5211
child: pid1 = 5212
parent: common = 161, addr = ffffeb7e78f4
child: common = 163, addr = ffffeb7e78f4
common = 171, addr = ffffeb7e78f4
common = 173, addr = ffffeb7e78f4
```

操作：在 return 0 之前对 common 加 10，观察输出。

分析：父进程和子进程各自维护变量的独立值，无论父进程还是子进程修改，均不会影响对方的结果。验证了每个进程拥有独立的数据段副本，进程退出时，其独立的地址空间被系统释放，不会对其他进程造成影响。体现了进程资源独立、地址空间隔离的核心思想。

步骤五：修改图 1-1 程序，在子进程中调用 system()与 exec 族函数。编写 system_call.c 文件输出进程号 PID，编译后生成 system_call 可执行文件。在子进程中调用 system_call，观察输出结果并分析总结。

使用 system():

```
[root@kp-test01 test]# gcc test1.c -o test
[root@kp-test01 test]# ./test
parent process PID: 2525
child process1 PID: 2526
system_call PID: 2527
child process PID: 2526
[root@kp-test01 test]# ./test
parent process PID: 2528
child process1 PID: 2529
system_call PID: 2530
child process PID: 2529
[root@kp-test01 test]# ./test
parent process PID: 2531
child process1 PID: 2532
system_call PID: 2533
child process PID: 2532
```

使用 exec():

```
[root@kp-test01 test]# gcc test1.c -o test
[root@kp-test01 test]# ./test
parent process PID: 2539
child process1 PID: 2540
system_call PID: 2540
[root@kp-test01 test]# ./test
parent process PID: 2541
child process1 PID: 2542
system_call PID: 2542
[root@kp-test01 test]# ./test
parent process PID: 2543
child process1 PID: 2544
system_call PID: 2544
```

分析：system()输出三个不同的 PID，是因为 system()内部再次 fork()出新进程执行命令；exec()输出的 system_call PID 与原子进程 PID 相同，是因为 exec()不创建新进程，而是用新程序替换当前子进程的进程映像。exec()后子进程被替换，因此不会输出原先进程的 pid。

1.6 线程相关编程实验

步骤一：设计程序，创建两个子线程，两线程分别对同一个共享变量多次操作，观察输出结果。

```
[root@kp-test01 test]# gcc -o thread thread.c -pthread
[root@kp-test01 test]# ./thread
Final value of shared_var = -25300
[root@kp-test01 test]# ./thread
Final value of shared_var = -1113100
[root@kp-test01 test]# ./thread
Final value of shared_var = 709100
```

操作：定义共享变量初始值为 0，两个线程分别对其进行 100000 次+/-100 操作，最终在主进程中输出处理后的变量值。

分析：结果不稳定，说明出现了竞争，两个线程几乎同时访问并修改同一个变量，有些操作结果没有正确地存储到寄存器中，从而导致最终结果随机。

步骤二：修改程序，定义信号量 signal，使用 PV 操作实现共享变量的访问与互斥。运行程序，观察最终共享变量的值。

```
[root@kp-test01 test]# gcc thread2.c -o thread -pthread
[root@kp-test01 test]# ./thread2
-bash: ./thread2: No such file or directory
[root@kp-test01 test]# ./thread
Final value of shared_var = 0
[root@kp-test01 test]# ./thread
Final value of shared_var = 0
[root@kp-test01 test]# ./thread
Final value of shared_var = 0
```

操作：设置 sem_t signal 作为互斥锁，在+/-前加入 P 操作申请资源，+/-后加入 V 操作释放资源。

分析：有效地避免了竞争的发生，使两线程的+/-操作正确运行。

步骤三：在第一部分实验了解了 system()与 exec 族函数的基础上，将这两个函数的调用改为在线程中实现，输出进程 PID 和线程的 TID 进行分析。

```
[root@kp-test01 test]# gcc thread3.c -o thread3 -pthread
[root@kp-test01 test]# ./thread3
[thread_system] PID = 2630, TID = 2631
system_call PID: 2632
[thread_exec] PID = 2630, TID = 2633
system_call PID: 2630
[root@kp-test01 test]# ./thread3
[thread_system] PID = 2634, TID = 2635
system_call PID: 2636
[thread_exec] PID = 2634, TID = 2637
system_call PID: 2634
[root@kp-test01 test]# ./thread3
[thread_system] PID = 2638, TID = 2639
system_call PID: 2640
[thread_exec] PID = 2638, TID = 2641
system_call PID: 2638
```

操作：创建两个线程，一个执行 system()，执行完毕（插入 sleep）后，另一个执行 exec()。

分析：system()在当前线程中新建进程执行命令，执行完返回；exec()在当前线程中替换整个进程映像；一旦执行 exec()，所有线程被销毁，只保留新程序。

1.7 自旋锁实验

根据实验内容要求，编写模拟自旋锁程序代码 spinlock.c：

```c
#include <stdio.h>
#include <pthread.h>
// 定义自旋锁结构体
typedef struct {
```

```
    int flag;
} spinlock_t;
// 初始化自旋锁
void spinlock_init(spinlock_t *lock) {
    lock->flag = 0;
}
// 获取自旋锁
void spinlock_lock(spinlock_t *lock) {
    while (__sync_lock_test_and_set(&lock->flag, 1)) {
// 自旋等待
    }
}
// 释放自旋锁
void spinlock_unlock(spinlock_t *lock) {
    __sync_lock_release(&lock->flag);
}
// 共享变量
int shared_value = 0;
// 线程函数
void *thread_function(void *arg) {
    spinlock_t *lock = (spinlock_t *)arg;
    for (int i = 0; i < 5000; ++i) {
        spinlock_lock(lock);
        shared_value++;
        spinlock_unlock(lock);
    }
    return NULL;
}
int main() {
    pthread_t thread1, thread2;
    spinlock_t lock;
    // 初始化自旋锁
    spinlock_init(&lock);
    // 输出初始共享变量
    printf("Shared value = %d\n", shared_value);
```

```
    // 创建两个线程
    pthread_create(&thread1, NULL, thread_function, &lock);
    printf("Thread1 creat success!\n");
    pthread_create(&thread2, NULL, thread_function, &lock);
    printf("Thread2 creat success!\n");
    // 等待线程结束
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    // 输出最终共享变量
    printf("Shared value = %d\n", shared_value);
    return 0;
}
```

运行结果：

```
[root@kp-test01 test]# gcc spinlock.c -o spinlock -lpthread
[root@kp-test01 test]# ./spinlock
Shared value = 0
Thread1 creat success!
Thread2 creat success!
Shared value = 10000
[root@kp-test01 test]# ./spinlock
Shared value = 0
Thread1 creat success!
Thread2 creat success!
Shared value = 10000
[root@kp-test01 test]# ./spinlock
Shared value = 0
Thread1 creat success!
Thread2 creat success!
Shared value = 10000
```

分析：本实验通过 pthread 创建两个线程，定义共享变量 shared_value 初值为 0，每个线程循环执行 5000 次 shared_value++操作。为了避免竞争条件，在进入临界区前后使用自定义的自旋锁进行加锁和解锁，保证同一时刻只有一个线程修改 shared_value。

自旋锁通过忙等的方式等待锁的释放，当 flag 被设置为 1 时，其他线程持续循环等待，不进行上下文切换；当锁被释放（flag=0）后，等待线程立即获得锁。它能有效保证多线程对共享资源的互斥访问，防止数据竞争，保证线程运行结果准确。

1.8 实验总结

1.8.1 实验中的问题与解决过程

在实验过程中，主要遇到了以下几个问题：

（1）进程同步问题：在进行父子进程输出时，未调用 wait()函数导致输出顺序不确定，甚至出现父进程先结束的情况。通过分析 wait()的作用，了解到它可以使父进程等待子进程结束，从而避免僵尸进程的产生。重新加入 wait()后问题解决。

（2）共享变量干扰问题：在多线程操作同一变量时，出现了多次运行结果不一致的情况。通过调试发现是线程间存在竞争条件，导致共享资源被同时修改。后采用信号量的 PV 操作及自旋锁机制，成功实现线程间的互斥访问，使结果稳定正确。

（3）system 与 exec 函数理解混淆：最初认为二者都能创建新进程执行外部程序，后通过实验发现，system()内部会再次 fork 出新进程执行命令，而 exec()则是直接替换当前进程映像。通过打印 PID 和父进程 PID，对两者的区别有了清晰认识。

1.8.2 实验收获

通过本次实验，我对 openEuler 系统环境及 Linux 进程线程模型有了更深入的理解：

（1）掌握了在 openEuler 环境下的基本操作与编译流程，了解了 Linux 系统中进程、线程的创建与管理机制。

（2）加深了对进程地址空间独立性的理解，认识到父子进程虽然变量名相同，但各自拥有独立的虚拟地址空间。

（3）通过信号量与自旋锁的实践，体会到操作系统中同步与互斥机制的重要性，学会了用不同方式解决多线程竞争问题。

（4）理解了 system()与 exec()的执行原理和区别，为后续学习进程控制与程序装载打下基础。

（5）整体实验增强了我对并发编程的感性认识，培养了调试与问题定位能力，也提高了在 Linux 环境下进行系统级实验的综合能力。

（6）通过调用系统库，我对以下函数有了更深入的认识：

a. fork()

- 功能：创建一个新的子进程。
- 返回值：在父进程中返回子进程 PID（>0），在子进程中返回 0，若失败返回 -1。
- 实验分析：父、子进程通过不同的返回值区分身份，能够实现并发执行。
  通过多次运行程序发现输出顺序不固定，说明父子进程调度由操作系统内核控制，是典型的并发执行现象。

b. getpid()

- 功能：获取当前进程的 PID（进程号）。
- 实验作用：在输出中用于验证父、子进程的区别与继承关系。子进程拥有独立的 PID。

c. getppid()

- 功能：获取当前进程的父进程 PID。
- 实验意义：可用于验证孤儿进程被 init 或 systemd 进程接管后的情况。

d. wait() / waitpid()

- 功能：使父进程阻塞，等待子进程结束并回收资源。
- 实验作用：在添加 wait() 后，父进程能确保子进程先执行完；去掉 wait() 后，父子进程可能并发输出，甚至产生孤儿或僵尸进程。
- 结论：wait() 是实现父子进程同步和防止僵尸进程的重要机制。

e. system()

- 功能：在当前进程中新建子进程执行 Shell 命令，内部实际调用 fork()、exec() 和 wait()。
- 实验表现：当子进程调用 system("./system_call") 时，会新建一个子进程执行 system_call 程序。
  输出显示 system_call 的 PID 与原子进程不同，说明 system() 内部确实新建了子进程。

f. exec() 族函数（如 execvp()）

- 功能：用指定的新程序替换当前进程映像。
- 实验现象：在子进程中调用 exec() 后，原程序被完全替换，PID 保持不变。

输出结果中可以看到 system_call 的 PID 与原子进程一致，证明当前进程被替换执行。

- 特点：exec() 调用后不会返回，除非执行失败。

g. pthread_create()

- 功能：创建一个新线程，并让其执行指定的函数。
- 实验作用：创建两个线程分别对共享变量进行加减操作，或在不同线程中调用 system() 与 exec() 函数。
- 分析：通过该函数实现了同一进程内的多线程并发执行。

h. pthread_self()与 syscall(SYS_gettid)

- 功能：获取当前线程的线程 ID（TID）。
- 实验作用：在输出中用于区分不同线程，分析同一进程下线程的唯一标识。syscall 可获取系统内核中的 tid，pthread_self 获取的是 pthread 库中的线程标识符。

i. pthread_join()

- 功能：等待指定线程结束，相当于线程间的同步机制。
- 实验现象：主线程调用 pthread_join() 后，会阻塞直到子线程执行完毕，否则主线程提前退出可能导致其他线程无法输出。
- 结论：pthread_join() 可防止主线程提前结束，是保证线程执行完整性的关键。

j. pthread_exit()

- 功能：终止当前线程并返回指定状态。
- 实验说明：虽然未在代码中显式调用，但线程函数结束时系统会隐式调用该函数。

k. pthread_mutex_init() / pthread_mutex_lock() / pthread_mutex_unlock()

- 功能：初始化互斥锁、上锁与解锁。
- 实验作用：用于线程间访问共享变量时的互斥控制。
- 分析：通过互斥锁可确保同一时间只有一个线程进入临界区，避免数据竞争。

l. sem_init()、sem_wait()、sem_post()

- 功能：分别用于信号量的初始化、P 操作和 V 操作。

- 实验作用：在线程访问共享变量时控制进入临界区的线程数量，实现同步。
- 分析：信号量机制可防止多个线程同时修改共享数据，从而保持结果稳定。

1.8.3 意见与建议

（1）实验指导书内容较为全面，但部分实验建议在文档中补充更多关于 GCC 内置原子操作的原理说明，方便理解底层实现。

（2）可以制作一些教程，帮助初次使用的同学快速掌握命令行操作与系统调用调试。

（3）实验过程中若能提供部分错误案例或调试指导，将更有助于理解并发问题的实质与解决思路。

1.9 附件

1.9.1 程序

进程相关编程实验：

步骤四：

```c
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid, pid1;
    /* A global variable */
    int common = 162;
    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        pid1 = getpid();
        printf("child: pid = %d\n", pid);   /* A */
        printf("child: pid1 = %d\n", pid1); /* B */
        printf("child: common = %d, addr = %lx\n", ++common, (unsigned long)&common); /* change common */
    }
    else { /* parent process */
        pid1 = getpid();
        printf("parent: pid = %d\n", pid);   /* C */
        printf("parent: pid1 = %d\n", pid1); /* D */
        printf("parent: common = %d, addr = %lx\n", --common, (unsigned long)&common); /* change common */
        // wait(NULL);
    }
    common += 10;
    printf("common = %d, addr = %lx\n", common, (unsigned long)&common);
    return 0;
}
```

（前三个步骤的代码在该步骤中均有体现，故略去。）

步骤五：

```c
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    pid_t pid, pid1;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed\n");
        return 1;
    }
    else if (pid == 0) { /* child process */
        pid1 = getpid();
        printf("child process1 PID: %d\n", pid1);

        // 使用 system()
        // system("./system_call");

        // 使用 execvp()
        char *args[] = {"./system_call", NULL};
        execvp(args[0], args);
        perror("exec failed");
        exit(1);
    }
    else { /* parent process */
        pid1 = getpid();
        printf("parent process PID: %d\n", pid1);
        wait(NULL);
    }

    return 0;
}
```

```c
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = getpid();
    printf("system_call PID: %d\n", pid);

    return 0;
}
```

（左为 test1.c，右为 system_call.c）

线程相关编程实验：

步骤一：（thread.c）

```c
#include <stdio.h>
#include <pthread.h>

#define COUNT 100000

int shared_var = 0; // 共享变量

void* add_func(void* arg) {
    for (int i = 0; i < COUNT; i++) {
        shared_var += 100;
    }
    return NULL;
}

void* sub_func(void* arg) {
    for (int i = 0; i < COUNT; i++) {
        shared_var -= 100;
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, add_func, NULL);
    pthread_create(&t2, NULL, sub_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final value of shared_var = %d\n", shared_var);

    return 0;
}

```

步骤二：

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define COUNT 100000

int shared_var = 0;
sem_t signal;

void* add_func(void* arg) {
    for (int i = 0; i < COUNT; i++) {
        sem_wait(&signal);   // P 操作：申请资源（进入临界区）
        shared_var += 100;
        sem_post(&signal);   // V 操作：释放资源（离开临界区）
    }
    return NULL;
}

void* sub_func(void* arg) {
    for (int i = 0; i < COUNT; i++) {
        sem_wait(&signal);   // P 操作
        shared_var -= 100;
        sem_post(&signal);   // V 操作
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    sem_init(&signal, 0, 1);

    pthread_create(&t1, NULL, add_func, NULL);
    pthread_create(&t2, NULL, sub_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final value of shared_var = %d\n", shared_var);

    sem_destroy(&signal);
    return 0;
}
```

步骤三：

```c
// 打印当前进程和线程信息
void print_ids(const char *msg) {
    pid_t pid = getpid();
    pthread_t tid = syscall(SYS_gettid);
    printf("[%s] PID = %d, TID = %lu\n", msg, pid, tid);
}

// 线程1：在子线程中调用 system()
void* thread_system(void* arg) {
    print_ids("thread_system");
    // 调用 system_call 程序
    system("./system_call");
    return NULL;
}

// 线程2：在子线程中调用 exec()
void* thread_exec(void* arg) {
    print_ids("thread_exec");
    char *args[] = {"./system_call", NULL};
    execvp(args[0], args);  // 替换当前线程所在的整个进程映像
    perror("exec failed");
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread_system, NULL);
    sleep(1); // 确保 thread_system 先运行
    pthread_create(&t2, NULL, thread_exec, NULL);
    sleep(1);
    pthread_join(t1, NULL);
    //pthread_join(t2, NULL);

    return 0;
}

```

1.9.2 Readme（运行结果图片略）

````
# Lab1  openEuler 系统环境实验

## 进程相关实验

**步骤 1 :**运行给定的程序，结果如下：

**步骤 2 :**删去 wait()后，运行结果如下：

可以看出父进程和子进程是并发执行的，父子进程输出顺序随机，在父进程中 `fork()` 的返回值是子进程的 pid，而在子进程中 `fork()` 的返回值是 0。

去除 `wait(NULL)` 后，由于父进程没有执行 `wait()`，父进程可以不再等待子进程，则可能先于子进程结束，导致子进程变为孤儿进程；若父进程未正确处理子进程退出，可能会产生“僵尸进程”。

**步骤 3 :**

设置全局变量

```c
int common = 162;
```

在父进程中：

```c
common++;
```
````

在子进程中：

```C
common--;
```

运行结果：

可以看到父进程和子进程中输出的值并不相同，即这两个 `common` 并不是同一个变量。但是它们的地址却一致，这是因为此地址仅为 `common` 在其各自的进程地址空间内的相对地址。而子进程是从父进程 `fork()` 而来的，因此他们的 `common` 的相对地址也一致。

步骤 4 :

在 return 前增加对全局变量 `common` 的操作

```c
common += 10;
```

运行结果：

可以看到父进程和子进程分别在`return`之前对各自的`value`进行了+10 操作。父进程和子进程各自维护变量的独立值，无论父进程还是子进程修改，均不会影响对方的结果。验证了每个进程拥有独立的数据段副本，进程退出时，其独立的地址空间被系统释放，不会对其他进程造成影响。

步骤 5 :在子进程中调用 system()与 exec 族函数

system_call.c:

```
pid_t pid = getpid();
pthread_t tid = pthread_self();
printf("[system_call] PID = %d, TID = %lu\n", pid, tid);
```

运行结果：

system()

****

```c
system("./system_call");
printf("child process PID: %d\n", pid1);
exit(1);
```**exec 函数**

****

```c
char *args[] = {"./system_call", NULL};
execvp(args[0], args);
printf("child process PID: %d\n", pid1);
perror("exec failed");
```

```
exit(1);
```

在子进程中开始处调用 `execvp("./system_call", NULL);`， 此处的 `"./system_call"` 的作用是输出进程的 pid 值。可以看到子进程直接去执行了 `./system_call`，不会执行源程序中打印 pid 的部分。这是因为当进程调用 `execvp`时该进程的用户空间和数据将被新程序替换。

使用`system()`调用时， `system(...)` 内实际上调用了一次 `fork()`,也就是子进程创建出来一个自己的子进程，即父进程的"**孙子进程**"去执行 `./system_call` (因为 `./system_call` 中输出的 pid 不同于子进程执行源程序时输出的 pid)。因此无论`system()`在子进程开始处或者末尾处调用，输出均相同。

线程相关实验

编译代码，由于调用 pthread.h，应在 gcc 末尾添加-pthread

```
gcc thread.c -o thread -pthread
```

步骤 1 ：创建两个线程，分别对同一个共享变量多次操作，观察输出结果。

创建两个线程，分别对共享变量进行 100000 次+/-100 操作。add_func 进行+100 操作，sub_fuc 进行-100 操作。

```
/* create the thread */
pthread_create(&t1, NULL, add_func, NULL);
pthread_create(&t2, NULL, sub_func, NULL);
```

调用 pthread_join(tid, NULL) 对两个线程进行阻塞等待，并在结束后分别打印 shared_var 的值。

```
pthread_join(t1, NULL);
pthread_join(t2, NULL);
// Thread 1
for(i = 0; i < 100000; i++) {
    shared_var += 100;
}
```

```
// Thread 2
for(i = 0; i < 100000; i++) {
    shared_var -= 100;
}
```

运行结果：

线程相关实验

编译代码，由于调用`pthread.h`，应在`gcc`末尾添加`-pthread`

```bash
gcc thread.c -o thread -pthread
```

步骤 1 ：创建两个线程，分别对同一个共享变量多次操作，观察输出结果。

创建两个线程，分别对共享变量进行 100000 次+/-100 操作。add_func 进行+100 操作，sub_fuc 进行-100 操作。

```c
/* create the thread */
pthread_create(&t1, NULL, add_func, NULL);
pthread_create(&t2, NULL, sub_func, NULL);
```

调用 `pthread_join(tid, NULL)` 对两个线程进行阻塞等待，并在结束后分别打印 `shared_var` 的值。

```c
pthread_join(t1, NULL);
```

```
pthread_join(t2, NULL);
```

```c
// Thread 1
for(i = 0; i < 100000; i++) {
    shared_var += 100;
}
// Thread 2
for(i = 0; i < 100000; i++) {
    shared_var -= 100;
}
```

运行结果：

可以观察到，多次运行程序，输出结果不同。与预期输出结果 0 并不一致。由于两个线程均要对`shared_var`的值进行修改，但是却没有实现互斥访问，造成结果输出错误。

**步骤 2: **定义信号量 signal，使用 PV 操作实现共享变量的访问与互斥。

定义初始化信号量

```C
sem_t signal; /* create signal */
...
// 互斥
sem_wait(&signal);   // P 操作：申请资源（进入临界区）
```

```
shared_var += 100;
sem_post(&signal);   // V 操作：释放资源（离开临界区）
```

运行结果：

可以观察到，实现了两个线程对共享变量的互斥访问，输出正确结果 0。

**步骤 3: **system()与 exec 族函数的基础上，将这两个函数的调用改为在线程中实现，输出进程 PID 。使用`syscall(SYS_gettid)`获取线程的 TID 进行分析。

观察结果可验证，`system()` 调用时在线程中创建新进程来运行被调用的外部程序，而`execvp()`调用时，直接将外部程序加载进当前进程，替换掉线程的后续执行代码。

自旋锁实验

补充主函数代码：

```
int main() {
    pthread_t thread1, thread2;
    spinlock_t lock;
    // 初始化自旋锁
    spinlock_init(&lock);
    // 输出初始共享变量
    printf("Shared value = %d\n", shared_value);
    // 创建两个线程
    pthread_create(&thread1, NULL, thread_function, &lock);
    printf("Thread1 creat success!\n");
    pthread_create(&thread2, NULL, thread_function, &lock);
```

```
    printf("Thread2 creat success!\n");
    // 等待线程结束
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    // 输出最终共享变量
    printf("Shared value = %d\n", shared_value);
    return 0;
}
```

运行结果：

可以观察到自旋锁用于在线程竞争共享资源的同步机制，输出结果与预期结果一致。使用自旋锁可以保证同一时间只有一个线程在对临界区进行操作，而其他线程处于忙等状态，保证了对临界区的互斥访问。

2 进程通信与内存管理

2.1 实验目的

（1）编程实现进程的创建和软中断通信，通过观察、分析实验现象，深入理解进程及进程在调度执行和内存空间等方面的特点，掌握在 POSIX 规范中系统调用的功能和使用。

（2）编程实现进程的管道通信，通过观察、分析实验现象，深入理解进程管道通信的特点，掌握管道通信的同步和互斥机制。

（3）通过设计实现内存分配管理的三种算法（FF，BF，WF），理解内存分配及回收的过程及实现思路，理解如何提高内存的分配效率和利用率。

（4）通过模拟实现页面置换算法（FIFO、LRU），理解请求分页系统中，页面置换的实现思路，理解命中率和缺页率的概念，理解程序的局部性原理，理解虚拟存储的原理。

2.2 实验内容

（1）编程实现进程的创建与软中断通信，使用 fork()、signal()、kill()、wait()、exit() 等系统调用，观察父进程与子进程间信号交互的过程。通过捕捉键盘中断信号（SIGINT、SIGQUIT），理解软中断通信的机制与执行流程。

（2）修改实验程序，通过改变中断信号类型，分析不同中断机制的执行方式与效果差异。

（3）编程实现进程间的管道通信，利用 pipe() 创建匿名管道，让两个子进程分别向管道写入数据，父进程从管道中读取数据，理解管道通信中的同步与互斥机制。

（4）比较管道通信中有锁与无锁情况下的运行结果，观察数据读写的差异与竞争问题，分析锁机制在通信同步中的作用。

（5）任选一项进行内存管理实验或页面置换实验：

① 内存管理：模拟首次适应（FF）、最佳适应（BF）和最坏适应（WF）三种算法，实现内存的动态分配、回收、合并与紧缩，分析不同算法的优缺点。

② 页面置换：模拟 FIFO 与 LRU 页面置换算法，计算缺页率，比较算法性能，分析局部性原理在页面置换中的体现。

2.3 实验思想

本实验围绕进程通信机制与内存管理原理展开，旨在通过程序设计与运行验证，理解操作系统在进程间协作与内存资源管理中的关键思想。

在软中断通信实验中，通过 fork() 创建子进程、signal() 捕捉信号、kill() 向进程发送信号的过程，深入理解进程的生命周期与信号驱动机制。通过捕捉键盘信号与闹钟信号的不同表现，掌握软中断机制的触发与响应原理。

在管道通信实验中，通过 pipe() 建立父子进程间的通信通道，模拟数据的写入与读取过程，理解管道通信的同步、互斥与进程阻塞唤醒机制。通过比较加锁与不加锁的运行结果，体会并发写入时的竞争现象，并掌握利用文件锁实现同步互斥的思想。

在内存与页面管理实验中，通过模拟 FF、BF、WF、FIFO、LRU 等算法，理解系统在动态内存分配与页面调度中的策略选择与效率权衡。通过观察算法在不同场景下的行为与性能，进一步掌握内存碎片、紧缩、页面命中率等核心概念，理解操作系统如何提高资源利用率与系统性能。

2.4 实验步骤

（1）在 VMWare 中配置 openEuler 虚拟机，尝试对虚拟机的使用。

（2）阅读 fork()、signal()、kill()、wait() 等函数的 man 手册，理解其功能与参数。

（3）根据实验指导书提供的软中断通信流程图（图 1）编写代码，实现父子进程间通过软中断信号进行通信。

（4）多次运行程序，分别在 5 秒内按下 Ctrl+C（SIGINT）和 Ctrl+\（SIGQUIT）键，或不按键，观察进程输出差异，并分析中断机制执行顺序。

（5）修改程序，将软中断信号改为 SIGALRM，利用 alarm() 函数触发闹钟中断，观察执行结果并与键盘中断进行对比分析。

（6）编写管道通信程序，利用 pipe() 创建管道，fork() 创建两个子进程分别写入不同字符，父进程读取输出。

（7）分别在有锁与无锁两种情况下运行程序，观察数据输出的顺序与完整性，分析锁机制在进程间同步中的作用。

（8）选择内存管理实验或页面置换实验：

① 内存管理实验：实现 FF、BF、WF 三种算法，编写菜单程序实现内存分配、释放与显示功能，运行并比较三种算法下的内存利用情况。

② 页面置换实验：实现 FIFO 与 LRU 算法，设置页面引用序列，输出页面置换过程与缺页率，分析不同算法的性能差异。

（9）记录实验过程与结果截图，分析各实验中不同策略下系统行为的差异，总结通信与内存管理机制的实现原理与思想。

2.5 进程的软中断通信

2.5.1 流程图

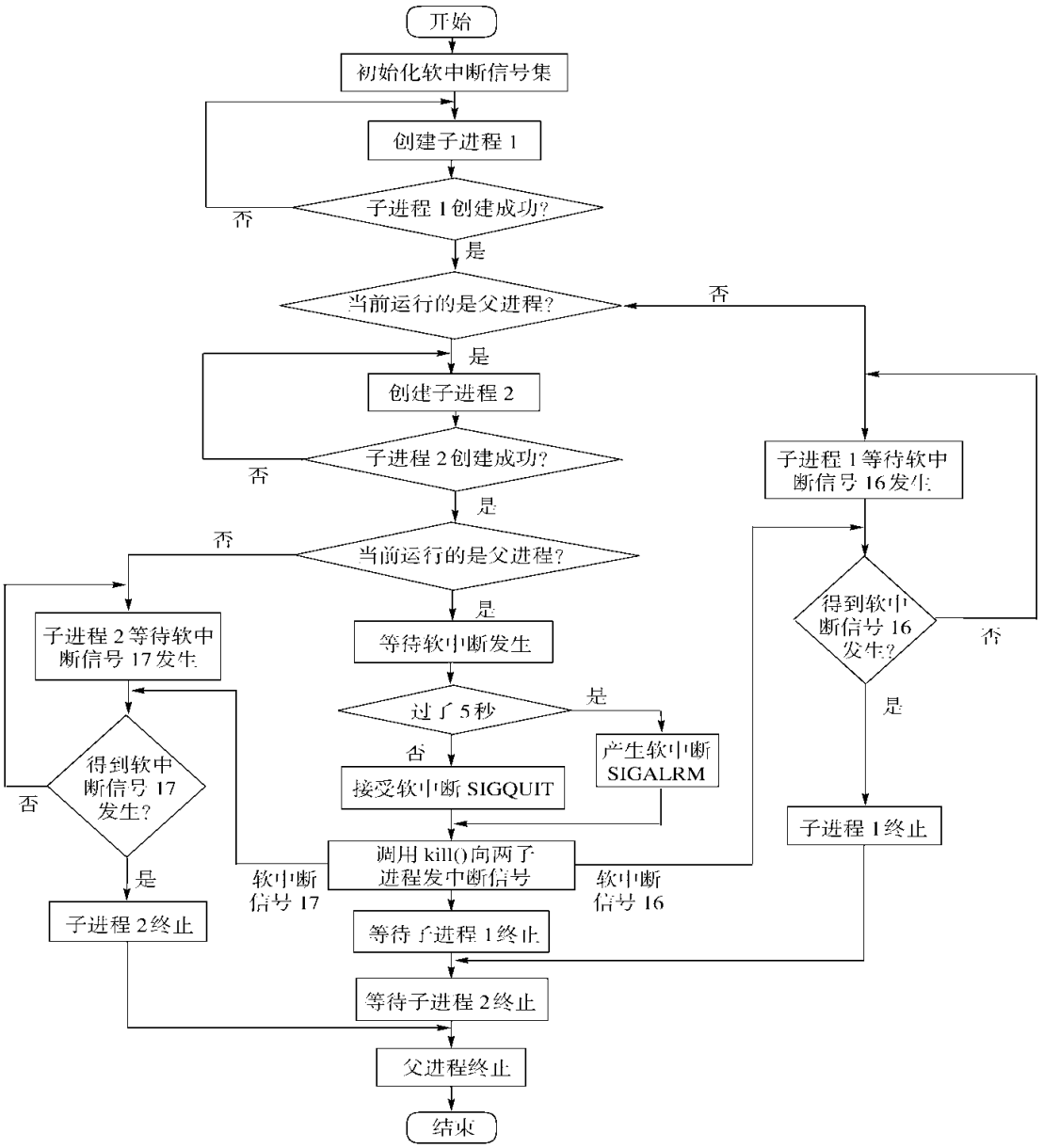
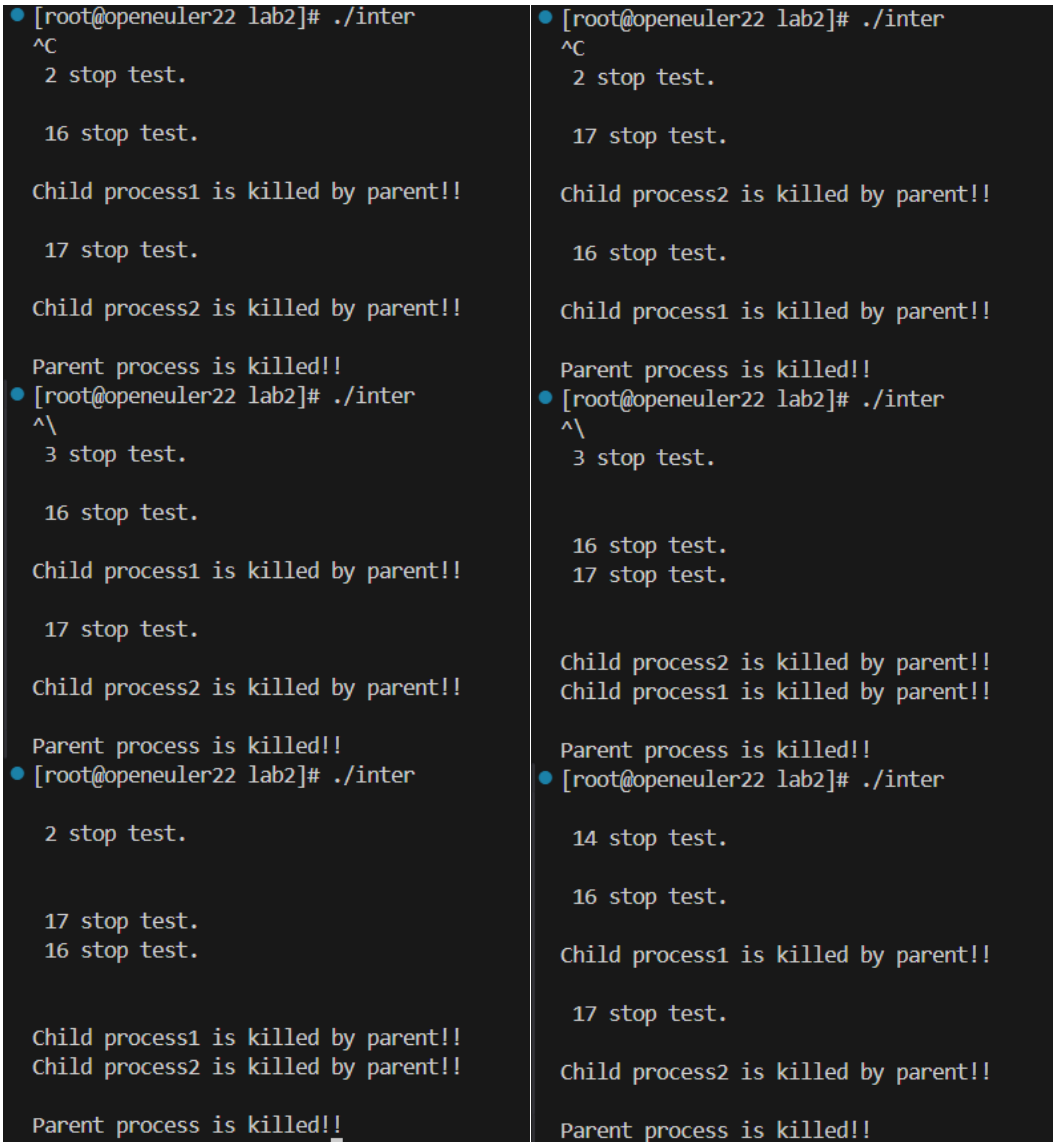


图 1　软中断通信流程图

2.5.2 运行结果：



（左为无闹钟信号，右为有闹钟信号）

图中，从上到下三次运行分别为 SIGINT、SIGQUIT 信号和五秒后中断。

2.5.3 问题分析

（1）你最初认为运行结果会怎么样？写出你猜测的结果。

答：在 5 秒内按下 Ctrl+C（产生 SIGINT）或 Ctrl+\（产生 SIGQUIT），父进程会捕获信号（inter_handler 设置 flag），随后父进程通过 kill 通知两个子进程结束；子进程收到信号后打印“Child processX is killed by parent!!”并退出；父进程在等待两个子进程结束后打印“Parent process is killed!!”并退出。如果在 5 秒内无键盘中断，waiting() 会在 sleep(5) 后使

父进程自发接收 SIGINT（通过 kill(getpid(), SIGINT)），从而触发与上面相同的流程。

（2）实际的结果什么样？有什么特点？在接收不同中断前后有什么差别？请将 5 秒内中断和 5 秒后中断的运行结果截图，试对产生该现象的原因进行分析。

实际结果：若按下 Ctrl+C（SIGINT）或 Ctrl+\（SIGQUIT），父进程捕获并进入 inter_handler，然后继续执行 waiting() 后续逻辑，通过 sleep(5) 实现等待并自杀逻辑；如果 flag 已被设置，可以跳过自杀分支，父进程随后的 kill(pid1, SIGSTKFLT) / kill(pid2, SIGCHLD) 会被执行以通知子进程。如果 5 秒内没有外部中断，父进程会在 waiting()后向自身发送 SIGINT，触发 inter_handler，继而运行父进程向子进程发送信号并等待子进程结束的流程。由于子进程对 SIGINT、SIGQUIT 设为 SIG_IGN，所以若父进程直接发送 SIGINT/SIGQUIT，子进程不会响应。但父选择向子发送它们各自登记的信号（子 1 的 16 号信号 SIGSTKFLT、子 2 的 17 号信号 SIGCHLD），因此子能在 pause() 中被唤醒，执行其处理函数并退出。

特点与差异原因：

①信号来源不同但处理流程一致：键盘中断由终端发送给父进程，父进程捕获后触发后续；超时则是父进程自己调用 kill(getpid(), SIGINT)；两者最后都通过父发向子进程的 kill(pid, ...) 来通知子进程结束，因此表面上观察到的父、子输出顺序是一致的。

②父–子进程对某些信号的忽略：子把 SIGINT、SIGQUIT 设为 SIG_IGN，这样在父进程接收到或自发产生这些信号时，若父不再转发子这些信号，则子不会被中断。代码通过向子发送特定信号确保子能被正确唤醒并退出。

（3）改为闹钟中断后，程序运行的结果是什么样子？与之前有什么不同？

结果：若将等待机制改为使用 alarm(5)，则父进程会在 5 秒后由内核发送 SIGALRM 到父进程，而不是父进程自己 kill(getpid(), SIGINT)。父在 inter_handler 中设置 flag，接着父会发送对子进程的结束信号并等待其退出，总体流程与自发 kill(getpid(), SIGINT)类似。

主要差别：

①信号来源不同：SIGALRM 是由内核定时器触发，更清晰可靠；而 kill(getpid(), SIGINT) 是程序自己向自身发送 SIGINT，语义上显得不自然。

②如果使用 SIGALRM，无需对子进程进行特殊信号语义映射；可以直接让父在 SIGALRM 处理器中执行通知子进程的逻辑。SIGALRM 避免了在代码里人为制造 SIGINT 的情况，从可读性与维护性上更优。

（4）kill 命令在程序中使用了几次？每次的作用是什么？执行后的现象是什么？

答：kill 被使用了三次：

①kill(getpid(), SIGINT);（位于 waiting() 中，仅在 5 秒超时且 flag == 0 时执行）

作用：程序向自身发送 SIGINT，触发父进程的 inter_handler；等价于“父进程自己产生一个中断”以进入信号处理分支。

执行后的现象：父进程进入 inter_handler（输出信号号、设置 flag），随后父进程继续执行发送给子进程的 kill 操作，唤醒/通知子进程退出。

②kill(pid1, SIGSTKFLT);（父在 waiting() 返回后对子进程 1 发）

作用：向子进程 1 发送 16 号信号，子进程 1 对该信号安装了 inter_handler，因此子进程 1 将从 pause() 唤醒并在处理器返回后打印“Child process1 is killed by parent!!”然后退出。

执行后的现象：子进程 1 被唤醒并退出；父通过 waitpid(pid1, ...) 等待其结束。

③kill(pid2, SIGCHLD);（父对子进程 2 发）

作用：向子进程 2 发送 17 号信号，子进程 2 对该信号安装了 inter_handler，因此子进程 2 会被唤醒并退出。

执行后的现象：子进程 2 打印“Child process2 is killed by parent!!”并退出；父通过 waitpid(pid2, ...) 等待其结束。

（5）使用 kill 命令可以在进程的外部杀死进程。进程怎样能主动退出？这两种退出方式哪种更好一些？

进程主动退出方法：调用 exit()、自然执行到末尾或通过父进程通知后由子

自行 exit()。

- 优点：可以做清理工作（释放资源、写日志、关闭文件描述符、通知其他进程等），允许程序以受控方式终止，避免资源泄露或部分写入。
- 缺点：若进程遇到不可恢复错误或被恶意阻止主动退出，可能无法按期终止。

外部 kill：

- 优点：能强制终止无法响应的进程（例如死锁或被阻塞的场景），管理员可以通过 SIGKILL 立即终止问题进程。
- 缺点：若使用强制信号，进程无法做善后处理；若使用非强制信号，需要进程正确处理该信号才能优雅退出；另外，误用或语义不当的信号会造成混淆。

结论：进程主动退出更好，因为其更安全、更可控。当进程不能主动终止或出现异常时，才使用外部 kill 强制结束。

2.6 进程的管道通信

2.6.1 流程图

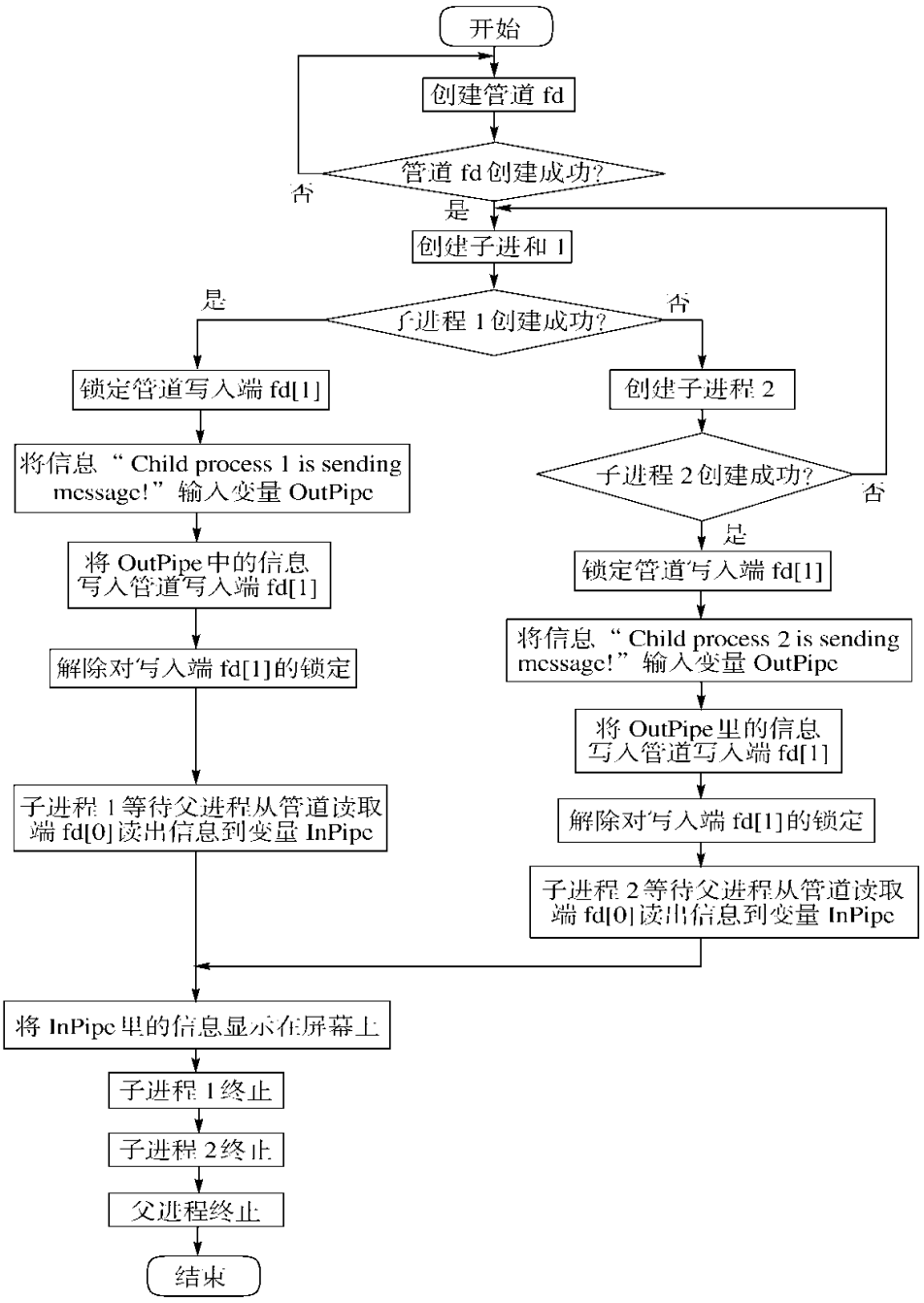


图 2　管道通信程序流程图

2.6.2 运行结果

（1）按流程图传输信息：

```
[root@openeuler22 lab2]# ./pipe
Child process 1 is sending message!
Child process 2 is sending message!

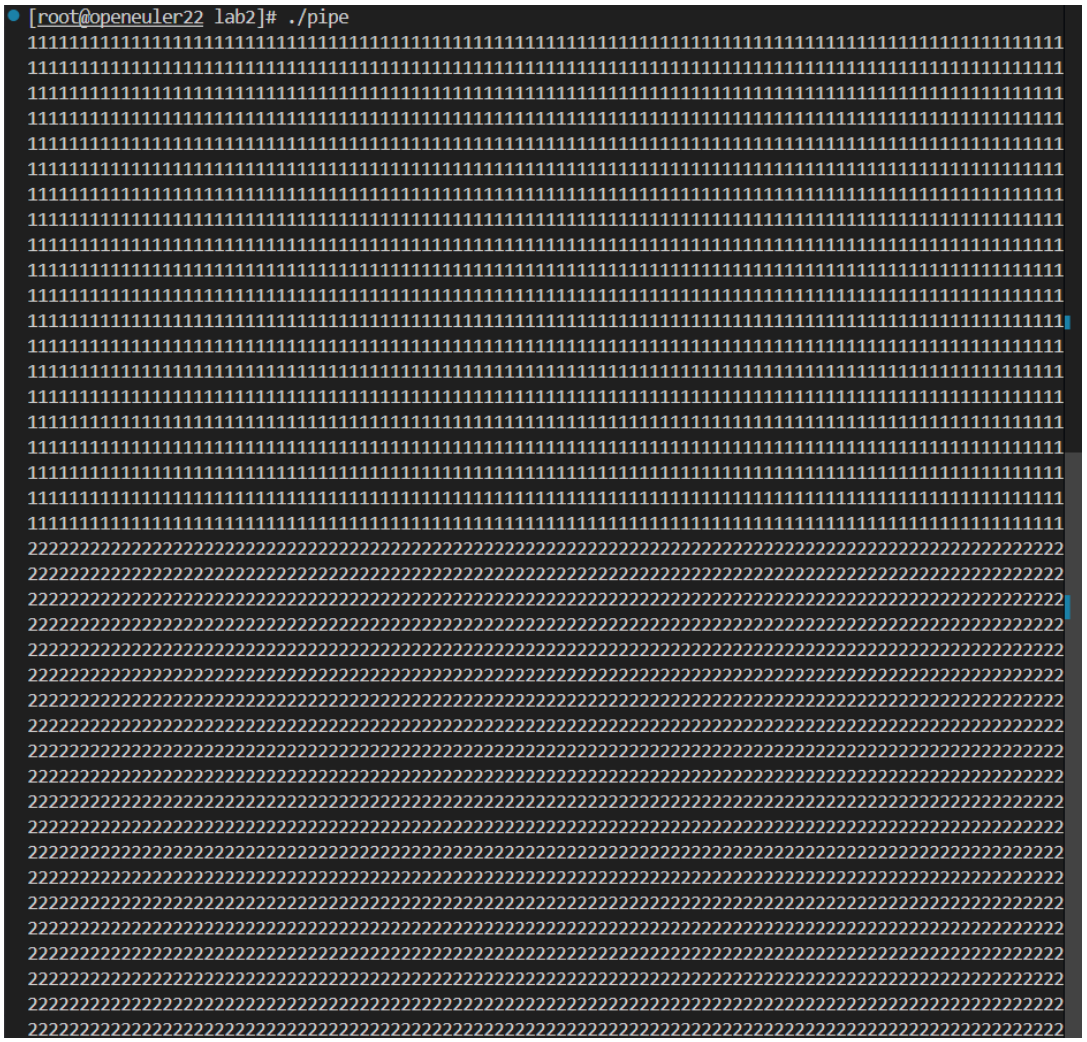
[root@openeuler22 lab2]# ./pipe
Child process 1 is sending message!
Child process 2 is sending message!

[root@openeuler22 lab2]# ./pipe
Child process 1 is sending message!
Child process 2 is sending message!

[root@openeuler22 lab2]# ./pipe
Child process 1 is sending message!
Child process 2 is sending message!
```

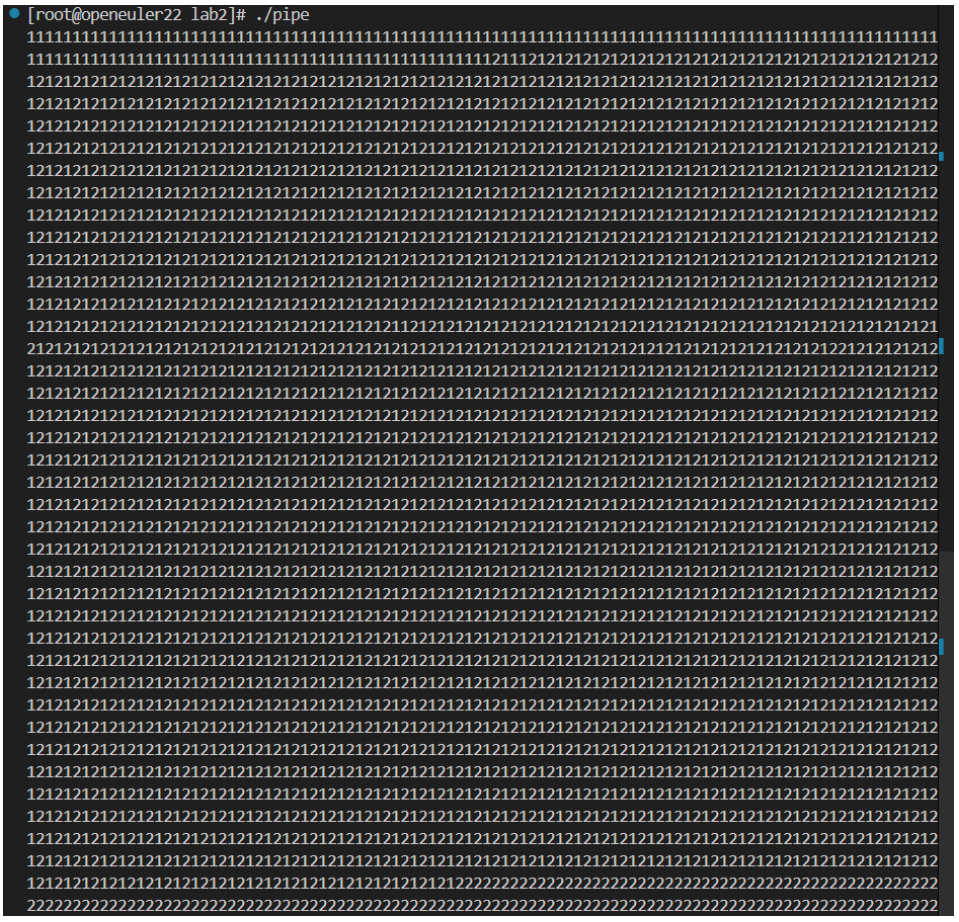
两个子进程向父进程发送的消息都被成功读取。

（2）加锁情况下两进程各写入 2000 个 1 或 2：



因为对写操作加锁，所以在进入子进程 1 后，连续写入 2000 个 1 后释放锁，子进程 2 才能开始写入 2000 个 2。在结果中，可以看到先将 1 读取完毕后才开始读取 2。

（3）无锁情况下两进程各写入 2000 个 1 或 2：



由于没有加锁，在子进程 2 创建后，会与子进程 1 抢夺对写入管道的使用权，此时就会出现 1、2 被交替写入的情况。

2.6.3 问题分析

(1) 你最初认为运行结果会怎么样？

答：如果代码在写操作上使用了锁，预期父进程读取到的 `InPipe` 将包含两个子进程的写入内容，且两段写入不会相互交错。如果取消锁并且两个子进程同时频繁写入，写操作可能会因为多个写者同时向同一管道写入导致交错或出现错位，造成读取到的内容不完整或顺序不确定。父进程在 `wait()` 等待子进程结束后再读，

避免了读取时写端仍打开导致读取阻塞，但若父先读而写端仍打开，read() 可能阻塞直到有足够数据或写端关闭。

(2)实际的结果什么样？有什么特点？试对产生该现象的原因进行分析。

实际结果：

①有锁情形：写入被序列化，父进程读取到的数据为“子进程 1 的消息 + 子进程 2 的消息”的拼接。当子写入短字符串"Child process X is sending message!"时，数据整段写入，不会被其他进程中断，因此父读到的字符串整洁且完整。

②无锁情形：若两个子进程都进行大量写入，写操作之间会交错或部分混合。因此不加锁会出现竞争，读取到的内容顺序不确定、可能被打断或混合，从而产生数据错乱或并发写入的污染。

③读阻塞与进程退出：父进程先 wait() 等待两个子退出，子退出后写端文件描述符全部关闭，read(fd[0], InPipe, 4000) 会读出所有管道中数据并返回，随后父打印并退出。这种先等待再读避免了读取阻塞的问题，因为写端已关闭，read 能立即返回已有数据。

(3)实验中管道通信是怎样实现同步与互斥的？如果不控制同步与互斥会发生什么后果？

实现方式：

①文件锁（lockf）：在写端上加锁以保证一次只有一个进程进行写入，从而实现互斥。适用于写入较大数据并需保证写入原子性时的序列化。

②通过父进程中使用的 wait(0) 系统调用，让父进程在执行 read 之前阻塞等待两个子进程的终止，确保数据在读取之前已被完整写入管道。

若不控制同步互斥会发生：

①数据交错/损坏：多个写者同时写入大于 PIPE_BUF 的数据会发生拆分与交错，导致读取端接收到混合数据。

②顺序不可预测：读取端无法保证接收到的消息顺序，因为不同子进程写入顺序不确定。

2.7 页面的置换

2.7.1 FIFO 算法

（1）流程图：

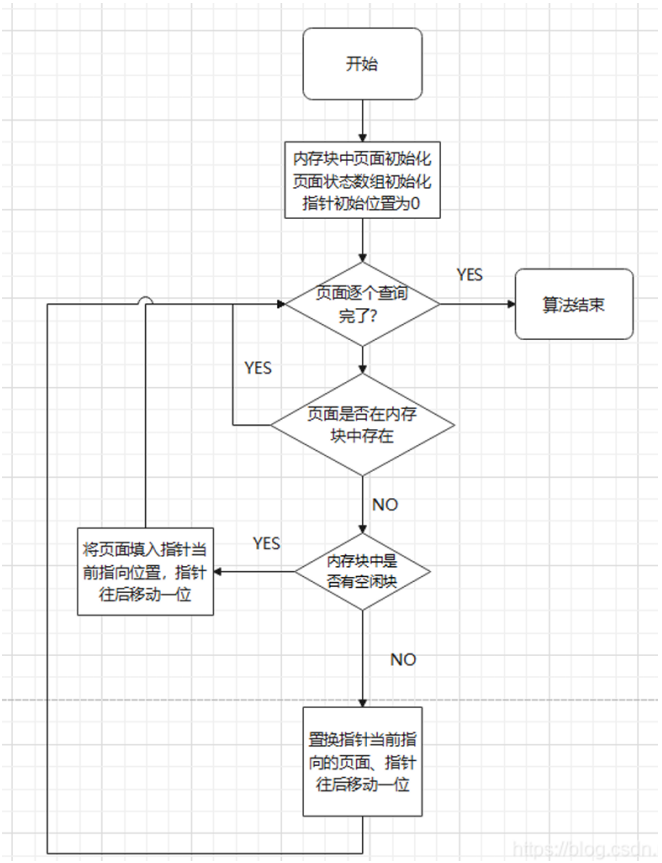
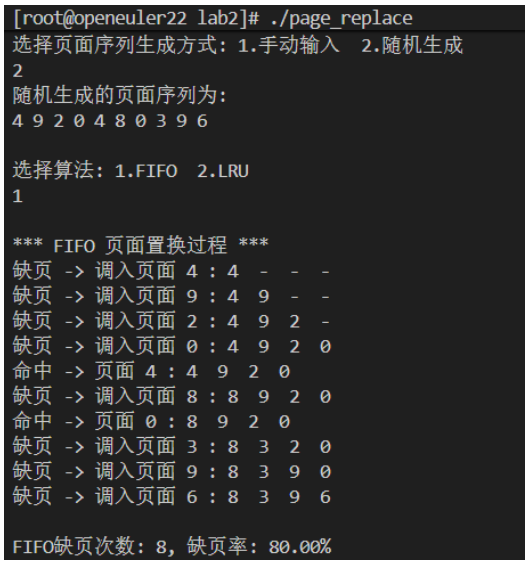


图 3　FIFO 算法流程图

（2）运行结果：





（3）设计方法：

使用 next_to_replace 指针循环替换页，每次命中检查后若缺页，将页面放入 frame[next_to_replace]并自增指针，实现页面队列的先进先出置换。

（4）Belady 现象：

设计页面序列 1，2，3，4，1，2，5，1，2，3，4，5，观察该序列在物理块数为 3 块和 4 块时的缺页情况：

```
[root@openeuler22 lab2]# ./page_replace
请输入页面引用序列长度：12
请输入物理块数量：3
选择页面序列生成方式：1.手动输入  2.随机生成
1
请输入页面引用序列（以空格分隔）：
1 2 3 4 1 2 5 1 2 3 4 5

选择算法：1.FIFO  2.LRU
1

*** FIFO 页面置换过程 ***
缺页 -> 调入页面 1：1  -  -
缺页 -> 调入页面 2：1  2  -
缺页 -> 调入页面 3：1  2  3
缺页 -> 调入页面 4：4  2  3
缺页 -> 调入页面 1：4  1  3
缺页 -> 调入页面 2：4  1  2
缺页 -> 调入页面 5：5  1  2
命中 -> 页面 1：5  1  2
命中 -> 页面 2：5  1  2
缺页 -> 调入页面 3：5  3  2
缺页 -> 调入页面 4：5  3  4
命中 -> 页面 5：5  3  4

FIFO缺页次数：9，缺页率：75.00%
```

```
[root@openeuler22 lab2]# ./page_replace
请输入页面引用序列长度：12
请输入物理块数量：4
选择页面序列生成方式：1.手动输入  2.随机生成
1
请输入页面引用序列（以空格分隔）：
1 2 3 4 1 2 5 1 2 3 4 5

选择算法：1.FIFO  2.LRU
1

*** FIFO 页面置换过程 ***
缺页 -> 调入页面 1：1  -  -  -
缺页 -> 调入页面 2：1  2  -  -
缺页 -> 调入页面 3：1  2  3  -
缺页 -> 调入页面 4：1  2  3  4
命中 -> 页面 1：1  2  3  4
命中 -> 页面 2：1  2  3  4
缺页 -> 调入页面 5：5  2  3  4
缺页 -> 调入页面 1：5  1  3  4
缺页 -> 调入页面 2：5  1  2  4
缺页 -> 调入页面 3：5  1  2  3
缺页 -> 调入页面 4：4  1  2  3
缺页 -> 调入页面 5：4  5  2  3

FIFO缺页次数：10，缺页率：83.33%
```

观察可得，物理块增加，缺页率反而提高，证明了 Belady 现象的存在。

2.7.2 LRU 算法

（1）流程图：

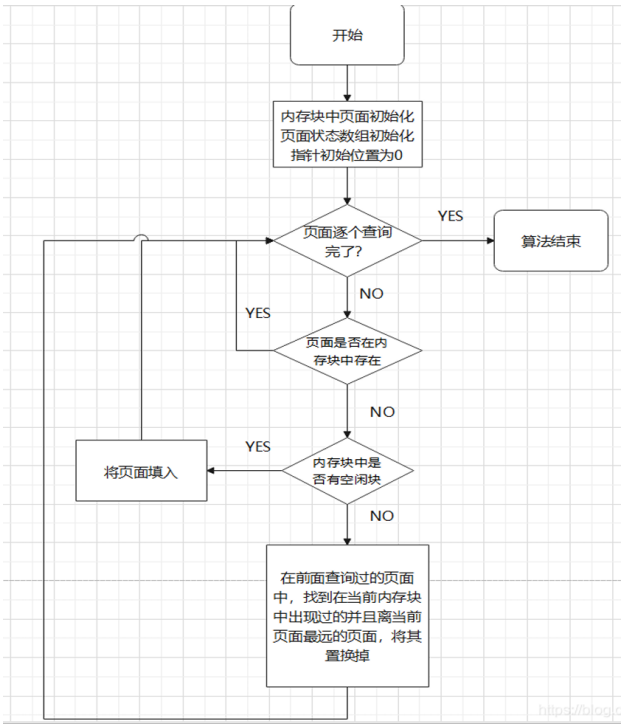


图 4　LRU 算法流程图

（2）运行结果：

```
[root@openeuler22 lab2]# ./page_replace
请输入页面引用序列长度：8
请输入物理块数量：5
选择页面序列生成方式：1.手动输入  2.随机生成
1
请输入页面引用序列（以空格分隔）：
1 2 3 1 4 5 6 9

选择算法：1.FIFO  2.LRU
2

*** LRU 页面置换过程 ***
缺页 -> 调入页面 1：-  1  -  -  -
缺页 -> 调入页面 2：-  1  2  -  -
缺页 -> 调入页面 3：-  1  2  3  -
命中 -> 页面 1：-  1  2  3  -
缺页 -> 调入页面 4：-  1  2  3  4
缺页 -> 调入页面 5：5  1  2  3  4
缺页 -> 调入页面 6：5  1  6  3  4
缺页 -> 调入页面 9：5  1  6  9  4

LRU缺页次数：7，缺页率：87.50%
```

```
[root@openeuler22 lab2]# ./page_replace
请输入页面引用序列长度：15
请输入物理块数量：6
选择页面序列生成方式：1.手动输入  2.随机生成
2
随机生成的页面序列为：
3 5 7 7 6 9 2 9 0 3 5 1 3 5 5

选择算法：1.FIFO  2.LRU
2

*** LRU 页面置换过程 ***
缺页 -> 调入页面 3：-  3  -  -  -  -
缺页 -> 调入页面 5：-  3  5  -  -  -
缺页 -> 调入页面 7：-  3  5  7  -  -
命中 -> 页面 7：-  3  5  7  -  -
缺页 -> 调入页面 6：-  3  5  7  6  -
缺页 -> 调入页面 9：-  3  5  7  6  9
缺页 -> 调入页面 2：2  3  5  7  6  9
命中 -> 页面 9：2  3  5  7  6  9
缺页 -> 调入页面 0：2  0  5  7  6  9
缺页 -> 调入页面 3：2  0  3  7  6  9
缺页 -> 调入页面 5：2  0  3  5  6  9
缺页 -> 调入页面 1：2  0  3  5  1  9
命中 -> 页面 3：2  0  3  5  1  9
命中 -> 页面 5：2  0  3  5  1  9
命中 -> 页面 5：2  0  3  5  1  9

LRU缺页次数：10，缺页率：66.67%
```

（3）设计方法：

使用 `last_used[]`数组记录每个帧最近被访问的时间，若缺页，在 `frame` 中寻找空闲帧优先使用，否则替换 `last_used` 最小（最久未使用）的帧。

2.7.3 问题分析

（1）从实现和性能方面，比较分析 FIFO 和 LRU 算法。

实现：

- FIFO：实现简单，维护一个替换指针 next_to_replace；每次缺页直接替换该指针指向的帧，时间复杂度低。
- LRU：实现更复杂，需要记录并维护每帧的最近使用时间，可使用列表数据结构实现以达到更优的时间复杂度。本代码中使用 last_used[] 并在缺页时扫描以找到最久未使用帧，整体时间复杂度与 FIFO 接近，但在更大规模实现下可以使用更复杂的数据结构使得更新与替换更高效。

性能：

- LRU 通常并且在大多数现实引用序列上优于 FIFO，因为 LRU 更贴合“最近被

使用的页面更可能再次被访问”的局部性原理，从而能保持近期使用的页面在内存中，减少缺页。

- FIFO 简单但可能抛弃近期仍会被访问的页面，在某些引用序列中产生较高的缺页率。有些情况下 FIFO 会出现异常，在增加帧数时缺页率反而上升，而 LRU 会更稳定。

（2）LRU 算法是基于程序的局部性原理而提出的算法，你模拟实现的 LRU 算法有没有体现出该特点？如果有，是如何实现的？

在 LRU 中，每当某个页面被访问，代码会把对应帧的 last_used 更新为当前时间 t；当需要替换时，选择 last_used 最小的帧（即最久未被访问的页面）进行替换。这样可以保留最近被访问的页面在内存中，从而利用时间局部性，减少再次缺页概率。因此，本实现从逻辑上体现了时间局部性：近期使用的页面被优先保留。

（3）在设计内存管理程序时,应如何提高内存利用率？

①使用高效的页面置换算法：采用 LRU 或其近似算法，以确保将最不可能再次使用的页面置换到磁盘，从而最小化缺页率，提高系统运行时间。

②分页：将物理内存划分为固定大小的块。这能有效解决外部碎片问题，并允许操作系统以更小的增量分配内存，从而高效地重新分配未使用的内存空间。

③分段：将程序划分为逻辑段，操作系统仅加载程序中需要的段，而不是整个程序。这有助于优化内存分配，但可能会增加运行时处理时间。

④虚拟内存与内存交换：允许运行大小超过物理内存容量的程序。当内存不足时，将应用程序内存临时从 RAM 转移到硬盘，从而释放物理内存空间。

⑤优化系统设置与避免内存泄漏：实施策略优化内存分配，并确保应用程序在使用完毕后释放未使用的内存，从而防止内存泄漏导致系统性能下降。

2.8 实验总结

2.8.1 实验中的问题与解决过程

在本次实验中，主要遇到了以下几个问题：

（1）信号同步问题：在实现进程软中断通信时，初始版本中未正确注册信号处

理函数，导致父进程发出信号后，子进程未能响应或提前退出。通过分析 signal() 的使用方法，了解到每个进程需要独立注册信号处理函数，才能正确捕获并处理信号。重新在父子进程中分别设置 signal() 后，通信得以正常实现。

（2）父子进程执行顺序混乱问题：在父进程使用 kill() 向子进程发送信号后，若未调用 wait() 等待子进程结束，会出现父进程先退出、子进程成为孤儿进程的情况。通过补充 waitpid() 回收子进程资源，保证了父进程在子进程完全结束后再退出，避免了僵尸进程的产生。

（3）管道通信数据竞争问题：在两个子进程同时向同一个管道写入数据时，初始版本未加锁，导致父进程读取的内容交错、输出乱码。通过增加 lockf(fd[1], 1, 0) 实现写端加锁机制，确保同一时刻只有一个子进程写入管道，从而解决了数据竞争问题。

（4）页面置换算法实现错误：在 FIFO 与 LRU 算法实现过程中，最初版本未正确初始化页面帧数组，导致替换逻辑错误和越界访问。通过在程序开始时对 frame[] 初始化为 -1，并正确维护 last_used[] 数组，成功修复算法行为，使得缺页统计与理论结果一致。

（5）LRU 访问时间更新问题：在 LRU 算法中，起初未在每次访问页面时更新 last_used[] 值，导致置换策略不准确。经调试后，在每次页面命中与缺页时均更新时间戳，使得最近最少使用页面被正确替换，程序输出的缺页次数合理。

2.8.2 实验收获

通过本次实验，我对 Linux 进程通信机制与内存页面管理策略有了系统的理解和实践体验，主要收获如下：

（1）掌握了进程间通信的两种方式：通过实验深入理解了信号通信与管道通信的原理与区别。信号机制适用于简单的事件通知，而管道通信可实现进程间有序的数据传递。实验让我熟悉了 signal()、kill()、pipe()、read()、write() 等系统调用的实际用法。

（2）加深了对进程同步与互斥的认识：通过使用 wait() 与 lockf() 等同步机制，理解了操作系统中进程间互斥的重要性。实验中我体会到，如果不控制访问顺序，进程可能会竞争资源或产生错误输出。

（3）理解了页面置换算法的原理与性能差异：通过实现 FIFO 与 LRU 两种算法，观

察到 FIFO 在循环访问模式下缺页率高，而 LRU 能够更好地利用局部性原理，显著减少缺页次数。实验中我首次通过程序验证了算法理论的正确性。

（4）掌握了系统调用的使用方法：实验让我熟悉了多种 Linux 系统调用的作用和返回值含义。尤其是：

- fork()：创建新进程并区分父子进程执行路径；
- wait() / waitpid()：实现进程同步并防止僵尸进程；
- kill()：用于进程间信号传递；
- signal()：定义进程对特定信号的响应行为；
- pipe()：建立匿名管道进行数据传输；
- lockf()：对文件描述符加锁，实现互斥访问。

（5）加深了对内存管理与局部性原理的理解：在 LRU 算法部分，通过记录页面最近一次访问时间，实现了“最近最少使用”置换策略，直观感受了程序局部性原理的实际效果。

（6）提高了调试与系统级编程能力：在多次实验调试中，逐步掌握了使用 printf、sleep 等工具追踪进程行为与系统调用流程的方法，增强了我在 Linux 环境下编程与问题定位的能力。

2.8.3 意见与建议

（1）实验指导书中关于信号类型及其触发机制的解释可以更详细，建议补充信号处理函数执行顺序与信号屏蔽机制的相关说明。

（2）在管道通信实验部分，可加入命名管道示例，帮助学生理解匿名管道与命名管道的区别。

（3）页面置换算法部分若能提供可视化页面访问过程的脚本或图示，将更直观地展示算法差异，帮助理解 LRU 与 FIFO 性能对比。

（4）实验建议提供标准测试输入与输出样例，便于学生验证程序逻辑的正确性与结果一致性。

2.9 附件

2.9.1 代码

1、进程的软中断通信

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <signal.h>
int flag = 0;
void inter_handler(int sig) {
    printf("\n %d stop test.\n", sig);
    flag = sig;
}
void waiting() {
    // alarm(5);
    // while (!flag) {
    //     pause();
    // }
    sleep(5);
    if (flag == 0) kill(getpid(), SIGINT);
}
int main() {
    // TODO: 五秒之后接收到两个信号
    pid_t pid1 = -1, pid2 = -1;
    while (pid1 == -1)
        pid1 = fork();
    if (pid1 > 0) {
        while (pid2 == -1)
            pid2 = fork();
        if (pid2 > 0) {
            // 父进程
            // SIGINT = 2, SIGQUIT = 3
            signal(SIGINT, inter_handler);
            signal(SIGQUIT, inter_handler);
            signal(SIGALRM, inter_handler);
            waiting();
```

```
            kill(pid1, SIGSTKFLT);
            kill(pid2, SIGCHLD);
            waitpid(pid1, NULL, 0);
            waitpid(pid2, NULL, 0);
            printf("\nParent process is killed!!\n");
        }
        else {
            // 子进程 2
            signal(SIGINT, SIG_IGN);
            signal(SIGQUIT, SIG_IGN);
            signal(SIGCHLD, inter_handler);
            pause();
            printf("\nChild process2 is killed by parent!!\n");
            return 0;
        }
    }
    else {
        // 子进程 1
        signal(SIGINT, SIG_IGN);
        signal(SIGQUIT, SIG_IGN);
        signal(SIGSTKFLT, inter_handler);
        pause();
        printf("\nChild process1 is killed by parent!!\n");
        return 0;
    }
    return 0;
}
```

2、进程的管道通信

```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <string.h>
int pid1, pid2; // 定义两个进程变量
int main() {
    int fd[2];
```

```
    char InPipe[4001]; // 定义读缓冲区
    char OutPipe[4001]; // 定义写缓冲区
    char c1 = '1', c2 = '2';
    if (pipe(fd) == -1) { // 创建管道
        perror("pipe");
        exit(1);
    }
    while ((pid1 = fork()) == -1); // 如果进程 1 创建不成功,则空循环
    // 如果子进程 1 创建成功,pid1 为进程号
    if (pid1 == 0) {
        lockf(fd[1], 1, 0);          // TODO:锁定管道
        // 将"Child process 1 is sending message!"写入 OutPipe
        snprintf(OutPipe, sizeof(OutPipe), "Child process 1 is sending
message!");
        write(fd[1], OutPipe, strlen(OutPipe));

        // TODO:分 2000 次每次向管道写入字符'1'
        // for (int i = 0; i < 2000; i++) {
        //     write(fd[1], &c1, 1);
        // }

        sleep(5); // 等待读进程读出数据
        lockf(fd[1], 0, 0);          // TODO:解除管道的锁定
        exit(0);  // 结束进程 1
    } else {
        while ((pid2 = fork()) == -1); // 若进程 2 创建不成功,则空循环
        if (pid2 == 0) {
            lockf(fd[1], 1, 0);
            // 将"Child process 2 is sending message!"写入 OutPipe
            snprintf(OutPipe, sizeof(OutPipe), "Child process 2 is sending
message!");
            write(fd[1], OutPipe, strlen(OutPipe));

            // TODO:分 2000 次每次向管道写入字符'2'
            // for (int i = 0; i < 2000; i++) {
            //     write(fd[1], &c2, 1);
```

```
            // }
            sleep(5);
            lockf(fd[1], 0, 0);
            exit(0);
        } else {
            wait(0);                // TODO:等待子进程 1 结束
            wait(0);                // 等待子进程 2 结束
            read(fd[0], InPipe, 4000);      // TODO:从管道中读出 4000 个字符
            InPipe[4000] = '\0';            //  TODO:加字符串结束符
            printf("%s\n", InPipe); // 显示读出的数据
            exit(0);                // 父进程结束
        }
    }
}
```

3、页面的置换

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX_PAGES 100    // 页面序列最大长度
#define MAX_FRAMES 10    // 物理块最大数量
int page_seq[MAX_PAGES]; // 页面访问序列
int frame[MAX_FRAMES];   // 当前内存块
int page_num;            // 页面序列长度
int frame_num;           // 内存块数量

// 打印当前内存状态
void print_frames() {
    for (int i = 0; i < frame_num; i++) {
        if (frame[i] == -1) printf(" - ");
        else printf(" %d ", frame[i]);
    }
    printf("\n");
}
```

```c
// FIFO 页面置换算法
void fifo() {
    int page_faults = 0;
    int next_to_replace = 0; // FIFO 替换指针

    for (int i = 0; i < frame_num; i++)
        frame[i] = -1;

    printf("\n*** FIFO 页面置换过程 ***\n");
    for (int i = 0; i < page_num; i++) {
        int page = page_seq[i];
        int found = 0;
        for (int j = 0; j < frame_num; j++) {
            if (frame[j] == page) {
                found = 1;
                break;
            }
        }

        if (!found) {
            frame[next_to_replace] = page;
            next_to_replace = (next_to_replace + 1) % frame_num;
            page_faults++;
            printf("缺页 -> 调入页面 %d :", page);
        } else {
            printf("命中 -> 页面 %d :", page);
        }
        print_frames();
    }

    printf("\nFIFO 缺页次数: %d, 缺页率: %.2f%%\n",
           page_faults, 100.0 * page_faults / page_num);
}

// LRU 页面置换算法
void lru() {
    int page_faults = 0;
```

```
    int last_used[MAX_FRAMES]; // 每个块最近使用的时间

    for (int i = 0; i < frame_num; i++) {
        frame[i] = -1;
        last_used[i] = -1;
    }

    printf("\n*** LRU 页面置换过程 ***\n");
    for (int t = 0; t < page_num; t++) {
        int page = page_seq[t];
        int found = 0;
        for (int j = 0; j < frame_num; j++) {
            if (frame[j] == page) {
                found = 1;
                last_used[j] = t; // 更新最近使用时间
                break;
            }
        }

        if (!found) {
            int replace_index = 0, oldest = last_used[0];
            for (int j = 1; j < frame_num; j++) {
                if (frame[j] == -1) { replace_index = j; break; }
                if (last_used[j] < oldest) {
                    oldest = last_used[j];
                    replace_index = j;
                }
            }
            frame[replace_index] = page;
            last_used[replace_index] = t;
            page_faults++;
            printf("缺页 -> 调入页面 %d :", page);
        } else {
            printf("命中 -> 页面 %d :", page);
        }
        print_frames();
    }
```

```
    printf("\nLRU 缺页次数: %d, 缺页率: %.2f%%\n",
           page_faults, 100.0 * page_faults / page_num);
}

int main() {
    int choice, mode;

    printf("请输入页面引用序列长度: ");
    scanf("%d", &page_num);
    printf("请输入物理块数量: ");
    scanf("%d", &frame_num);

    printf("选择页面序列生成方式: 1.手动输入  2.随机生成\n");
    scanf("%d", &mode);

    if (mode == 1) {
        printf("请输入页面引用序列（以空格分隔）:\n");
        for (int i = 0; i < page_num; i++)
            scanf("%d", &page_seq[i]);
    } else {
        srand(time(NULL));
        for (int i = 0; i < page_num; i++)
            page_seq[i] = rand() % 10; // 0~9 的随机页号
        printf("随机生成的页面序列为:\n");
        for (int i = 0; i < page_num; i++)
            printf("%d ", page_seq[i]);
        printf("\n");
    }

    printf("\n 选择算法: 1.FIFO  2.LRU\n");
    scanf("%d", &choice);

    if (choice == 1) fifo();
    else if (choice == 2) lru();
    else printf("无效选择。\n");
```

```
    return 0;
}
```

2.9.2 Readme

Lab2 进程通信与内存管理

软中断通信

编写程序：

```c++
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <signal.h>
int flag = 0;
void inter_handler(int sig) {
    printf("\n %d stop test.\n", sig);
    flag = sig;
}
void waiting() {
    // alarm(5);
    // while (!flag) {
    //     pause();
    // }
    sleep(5);
    if (flag == 0) kill(getpid(), SIGINT);
}
```

```
int main() {
    // TODO: 五秒之后接收到两个信号
    pid_t pid1 = -1, pid2 = -1;
    while (pid1 == -1)
        pid1 = fork();
    if (pid1 > 0) {
        while (pid2 == -1)
            pid2 = fork();
        if (pid2 > 0) {
            // 父进程
            // SIGINT = 2, SIGQUIT = 3
            signal(SIGINT, inter_handler);
            signal(SIGQUIT, inter_handler);
            signal(SIGALRM, inter_handler);
            waiting();
            kill(pid1, SIGSTKFLT);
            kill(pid2, SIGCHLD);
            waitpid(pid1, NULL, 0);
            waitpid(pid2, NULL, 0);
            printf("\nParent process is killed!!\n");
        }
        else {
            // 子进程 2
            signal(SIGINT, SIG_IGN);
            signal(SIGQUIT, SIG_IGN);
            signal(SIGCHLD, inter_handler);
            pause();
            printf("\nChild process2 is killed by parent!!\n");
            return 0;
```

```
        }
    }
    else {
        // 子进程 1
        signal(SIGINT, SIG_IGN);
        signal(SIGQUIT, SIG_IGN);
        signal(SIGSTKFLT, inter_handler);
        pause();
        printf("\nChild process1 is killed by parent!!\n");
        return 0;
    }
    return 0;
}
```

其中，

```C
signal(SIGQUIT, SIG_IGN);
signal(SIGINT, SIG_IGN);
signal(SIGALRM, SIG_IGN);
```

用于在子进程中忽略`SIGQUIT`、`SIGINT`、`SIGALRM`信号。

（1）你最初认为运行结果会怎么样？写出你猜测的结果。

答：在 5 秒内按下 Ctrl+C（产生 SIGINT）或 Ctrl+\（产生 SIGQUIT），父进程

会捕获信号（inter_handler 设置 flag），随后父进程通过 kill 通知两个子进程结束；子进程收到信号后打印“Child processX is killed by parent!!”并退出；父进程在等待两个子进程结束后打印“Parent process is killed!!”并退出。如果在 5 秒内无键盘中断，waiting() 会在 sleep(5) 后使父进程自发接收 SIGINT（通过 kill(getpid(), SIGINT)），从而触发与上面相同的流程。

（2）实际的结果什么样？有什么特点？在接收不同中断前后有什么差别？请将 5 秒内中断和 5 秒后中断的运行结果截图，试对产生该现象的原因进行分析。

实际结果：若按下 Ctrl+C（SIGINT）或 Ctrl+\（SIGQUIT），父进程捕获并进入 inter_handler，然后继续执行 waiting() 后续逻辑，通过 sleep(5) 实现等待并自杀逻辑；如果 flag 已被设置，可以跳过自杀分支，父进程随后的 kill(pid1, SIGSTKFLT) / kill(pid2, SIGCHLD) 会被执行以通知子进程。如果 5 秒内没有外部中断，父进程会在 waiting()后向自身发送 SIGINT，触发 inter_handler，继而运行父进程向子进程发送信号并等待子进程结束的流程。由于子进程对 SIGINT、SIGQUIT 设为 SIG_IGN，所以若父进程直接发送 SIGINT/SIGQUIT，子进程不会响应。但父选择向子发送它们各自登记的信号（子 1 的 16 号信号 SIGSTKFLT、子 2 的 17 号信号 SIGCHLD），因此子能在 pause() 中被唤醒，执行其处理函数并退出。

特点与差异原因：

①信号来源不同但处理流程一致：键盘中断由终端发送给父进程，父进程捕获后触发后续；超时则是父进程自己调用 kill(getpid(), SIGINT)；两者最后都通过父发向子进程的 kill(pid, ...) 来通知子进程结束，因此表面上观察到的父、子输出顺序是一致的。

②父—子进程对某些信号的忽略：子把 SIGINT、SIGQUIT 设为 SIG_IGN，这样在父

进程接收到或自发产生这些信号时，若父不再转发子这些信号，则子不会被中断。代码通过向子发送特定信号确保子能被正确唤醒并退出。

（3）改为闹钟中断后，程序运行的结果是什么样子？与之前有什么不同？

结果：若将等待机制改为使用 alarm(5)，则父进程会在 5 秒后由内核发送 SIGALRM 到父进程，而不是父进程自己 kill(getpid(), SIGINT)。父在 inter_handler 中设置 flag，接着父会发送对子进程的结束信号并等待其退出，总体流程与“自发 kill(getpid(), SIGINT)”类似。

主要差别：

①信号来源不同：SIGALRM 是由内核定时器触发，更清晰可靠；而 kill(getpid(), SIGINT) 是程序自己向自身发送 SIGINT，语义上显得不自然。

②如果使用 SIGALRM，无需对子进程进行特殊信号语义映射；可以直接让父在 SIGALRM 处理器中执行通知子进程的逻辑。SIGALRM 避免了在代码里人为制造 SIGINT 的情况，从可读性与维护性上更优。

（4）kill 命令在程序中使用了几次？每次的作用是什么？执行后的现象是什么？

答：kill 被使用了三次：

①kill(getpid(), SIGINT);（位于 waiting() 中，仅在 5 秒超时且 flag == 0 时执行）

作用：程序向自身发送 SIGINT，触发父进程的 inter_handler；等价于“父进程自己产生一个中断”以进入信号处理分支。

执行后的现象：父进程进入 inter_handler（输出信号号、设置 flag），随后父进程继续执行发送给子进程的 kill 操作，唤醒/通知子进程退出。

②kill(pid1, SIGSTKFLT);（父在 waiting() 返回后对子进程 1 发）

作用：向子进程 1 发送 16 号信号，子进程 1 对该信号安装了 inter_handler，因此子进程 1 将从 pause() 唤醒并在处理器返回后打印“Child process1 is killed by parent!!”然后退出。

执行后的现象：子进程 1 被唤醒并退出；父通过 waitpid(pid1, ...) 等待其结束。

③kill(pid2, SIGCHLD);（父对子进程 2 发）

作用：向子进程 2 发送 17 号信号，子进程 2 对该信号安装了 inter_handler，因此子进程 2 会被唤醒并退出。

执行后的现象：子进程 2 打印“Child process2 is killed by parent!!”并退出；父通过 waitpid(pid2, ...) 等待其结束。

（5）使用 kill 命令可以在进程的外部杀死进程。进程怎样能主动退出？这两种退出方式哪种更好一些？

进程主动退出方法：调用 exit()、自然执行到末尾或通过父进程通知后由子自行 exit()。

- 优点：可以做清理工作（释放资源、写日志、关闭文件描述符、通知其他进程等），允许程序以受控方式终止，避免资源泄露或部分写入。
- 缺点：若进程遇到不可恢复错误或被恶意阻止主动退出，可能无法按期终止。

外部 kill：

- 优点：能强制终止无法响应的进程（例如死锁或被阻塞的场景），管理员可以通过 SIGKILL 立即终止问题进程。
- 缺点：若使用强制信号，进程无法做善后处理；若使用非强制信号，需要进程正确处理该信号才能优雅退出；另外，误用或语义不当的信号会造成混淆。

结论：进程主动退出更好，因为其更安全、更可控。当进程不能主动终止或出现异常时，才使用外部 kill 强制结束。

管道通信

根据所给代码补充完在父(子)进程中读写的部分代码

```C
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
```

```
#include <string.h>
int pid1, pid2; // 定义两个进程变量
int main() {
    int fd[2];
    char InPipe[4001]; // 定义读缓冲区
    char OutPipe[4001]; // 定义写缓冲区
    char c1 = '1', c2 = '2';
    if (pipe(fd) == -1) { // 创建管道
        perror("pipe");
        exit(1);
    }
    while ((pid1 = fork()) == -1); // 如果进程 1 创建不成功,则空循环
    // 如果子进程 1 创建成功,pid1 为进程号
    if (pid1 == 0) {
        lockf(fd[1], 1, 0);          // TODO:锁定管道
        // 将"Child process 1 is sending message!"写入 OutPipe
        snprintf(OutPipe, sizeof(OutPipe), "Child process 1 is sending
message!");
        write(fd[1], OutPipe, strlen(OutPipe));

        // TODO:分 2000 次每次向管道写入字符' 1'
        // for (int i = 0; i < 2000; i++) {
        //     write(fd[1], &c1, 1);
        // }

        sleep(5); // 等待读进程读出数据
        lockf(fd[1], 0, 0);          // TODO:解除管道的锁定
        exit(0);  // 结束进程 1
    } else {
```

```
        while ((pid2 = fork()) == -1); // 若进程 2 创建不成功,则空循环
        if (pid2 == 0) {
            lockf(fd[1], 1, 0);
            // 将"Child process 2 is sending message!"写入 OutPipe
            snprintf(OutPipe, sizeof(OutPipe), "Child process 2 is
sending message!");
            write(fd[1], OutPipe, strlen(OutPipe));

            // TODO:分 2000 次每次向管道写入字符’2’
            // for (int i = 0; i < 2000; i++) {
            //     write(fd[1], &c2, 1);
            // }
            sleep(5);
            lockf(fd[1], 0, 0);
            exit(0);
        } else {
            wait(0);                // TODO:等待子进程 1 结束
            wait(0);                // 等待子进程 2 结束
            read(fd[0], InPipe, 4000);     // TODO:从管道中读出 4000 个字
符
            InPipe[4000] = '\0';           //  TODO:加字符串结束符
            printf("%s\n", InPipe); // 显示读出的数据
            exit(0);                // 父进程结束
        }
    }
}
```

在子进程中，删除 lockf 函数

```C
//lockf(fd[1], 1, 0);
、、、
//lockf(fd[1], 0, 0);
```

则管道读写的互斥和同步出现问题，加上之后输出结果正确。

(1)你最初认为运行结果会怎么样？

答：如果代码在写操作上使用了锁，预期父进程读取到的 InPipe 将包含两个子进程的写入内容，且两段写入不会相互交错。如果取消锁并且两个子进程同时频繁写入，写操作可能会因为多个写者同时向同一管道写入导致交错或出现错位，造成读取到的内容不完整或顺序不确定。父进程在 wait() 等待子进程结束后再读，避免了读取时写端仍打开导致读取阻塞，但若父先读而写端仍打开，read() 可能阻塞直到有足够数据或写端关闭。

(2)实际的结果什么样？有什么特点？试对产生该现象的原因进行分析。

实际结果：

①有锁情形：写入被序列化，父进程读取到的数据为“子进程 1 的消息 + 子进程 2 的消息”的拼接。当子写入短字符串"Child process X is sending message!"时，数据整段写入，不会被其他进程中断，因此父读到的字符串整洁且完整。

②无锁情形：若两个子进程都进行大量写入，写操作之间会交错或部分混合。因此不加锁会出现竞争，读取到的内容顺序不确定、可能被打断或混合，从而产生数据错乱或并发写入的污染。

③读阻塞与进程退出：父进程先 wait() 等待两个子退出，子退出后写端文件描述符全部关闭，read(fd[0], InPipe, 4000) 会读出所有管道中数据并返回，随后父打印并退出。这种先等待再读避免了读取阻塞的问题，因为写端已关闭，read 能立即返回已有数据。

(3)实验中管道通信是怎样实现同步与互斥的？如果不控制同步与互斥会发生什么后果？

实现方式：

①文件锁（lockf）：在写端上加锁以保证一次只有一个进程进行写入，从而实现互斥。适用于写入较大数据并需保证写入原子性时的序列化。

②通过父进程中使用的 wait(0) 系统调用，让父进程在执行 read 之前阻塞等待两个子进程的终止，确保数据在读取之前已被完整写入管道。

若不控制同步互斥会发生：

①数据交错/损坏：多个写者同时写入大于 PIPE_BUF 的数据会发生拆分与交错，导致读取端接收到混合数据。

②顺序不可预测：读取端无法保证接收到的消息顺序，因为不同子进程写入顺序不确定。

页面置换

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX_PAGES 100    // 页面序列最大长度
#define MAX_FRAMES 10    // 物理块最大数量
int page_seq[MAX_PAGES]; // 页面访问序列
int frame[MAX_FRAMES];   // 当前内存块
int page_num;            // 页面序列长度
int frame_num;           // 内存块数量

// 打印当前内存状态
void print_frames() {
    for (int i = 0; i < frame_num; i++) {
        if (frame[i] == -1) printf(" - ");
        else printf(" %d ", frame[i]);
    }
    printf("\n");
}

// FIFO 页面置换算法
void fifo() {
    int page_faults = 0;
    int next_to_replace = 0; // FIFO 替换指针
```

```
    for (int i = 0; i < frame_num; i++)
        frame[i] = -1;

    printf("\n*** FIFO 页面置换过程 ***\n");
    for (int i = 0; i < page_num; i++) {
        int page = page_seq[i];
        int found = 0;
        for (int j = 0; j < frame_num; j++) {
            if (frame[j] == page) {
                found = 1;
                break;
            }
        }

        if (!found) {
            frame[next_to_replace] = page;
            next_to_replace = (next_to_replace + 1) % frame_num;
            page_faults++;
            printf("缺页 -> 调入页面 %d :", page);
        } else {
            printf("命中 -> 页面 %d :", page);
        }
        print_frames();
    }

    printf("\nFIFO 缺页次数: %d, 缺页率: %.2f%%\n",
           page_faults, 100.0 * page_faults / page_num);
}
```

```
// LRU 页面置换算法
void lru() {
    int page_faults = 0;
    int last_used[MAX_FRAMES]; // 每个块最近使用的时间

    for (int i = 0; i < frame_num; i++) {
        frame[i] = -1;
        last_used[i] = -1;
    }

    printf("\n*** LRU 页面置换过程 ***\n");
    for (int t = 0; t < page_num; t++) {
        int page = page_seq[t];
        int found = 0;
        for (int j = 0; j < frame_num; j++) {
            if (frame[j] == page) {
                found = 1;
                last_used[j] = t; // 更新最近使用时间
                break;
            }
        }

        if (!found) {
            int replace_index = 0, oldest = last_used[0];
            for (int j = 1; j < frame_num; j++) {
                if (frame[j] == -1) { replace_index = j; break; }
                if (last_used[j] < oldest) {
                    oldest = last_used[j];
```

```
                    replace_index = j;
                }
            }
            frame[replace_index] = page;
            last_used[replace_index] = t;
            page_faults++;
            printf("缺页 -> 调入页面 %d :", page);
        } else {
            printf("命中 -> 页面 %d :", page);
        }
        print_frames();
    }

    printf("\nLRU 缺页次数: %d, 缺页率: %.2f%%\n",
           page_faults, 100.0 * page_faults / page_num);
}

int main() {
    int choice, mode;

    printf("请输入页面引用序列长度: ");
    scanf("%d", &page_num);
    printf("请输入物理块数量: ");
    scanf("%d", &frame_num);

    printf("选择页面序列生成方式: 1.手动输入  2.随机生成\n");
    scanf("%d", &mode);

    if (mode == 1) {
```

```
        printf("请输入页面引用序列（以空格分隔）:\n");
        for (int i = 0; i < page_num; i++)
            scanf("%d", &page_seq[i]);
    } else {
        srand(time(NULL));
        for (int i = 0; i < page_num; i++)
            page_seq[i] = rand() % 10; // 0~9 的随机页号
        printf("随机生成的页面序列为:\n");
        for (int i = 0; i < page_num; i++)
            printf("%d ", page_seq[i]);
        printf("\n");
    }

    printf("\n 选择算法: 1.FIFO  2.LRU\n");
    scanf("%d", &choice);

    if (choice == 1) fifo();
    else if (choice == 2) lru();
    else printf("无效选择。\n");

    return 0;
}
```

结果：

1.FIFO: 当需要调入页面到内存时，选择最先调入页面的那个置换掉。

2.LRU: 当需要调入页面到内存时，选择最近最少使用的页面置换掉。

3.当内存页面数增加了以后，命中率反而降低了。

4.具有局部性良好的数据，LRU 命中率高于 FIFO 算法

结果解释：

1.使用 FIFO 算法，创建一个队列来管理所有的内存页面，置换的是队列的首个页面。当需要调入页面到内存时，就将它加到队列的尾部。

2.LRU 置换将每个页面与它的上次使用的时间关联起来。当需要置换页面时，LRU 选择最长时间没有使用的页面。代码中为每个页表条目关联一个使用时间域 clock，并为 CPU 添加一个全局的计数器，每次内存引用都会递增时钟，并将时钟寄存器的内容复制到对应页表条目的使用时间域，这样每次最小的时钟域就是要被置换的页面。

3.Belady 异常指的是， 对于 FIFO 算法，随着内存分配帧数量的增加，命中率可能会下降。

4.LRU 具有良好的局部性，对于最近出现过的页面，更不可能被换出，而局部性数据一般会多次重复访问，所以命中率会更高。

3 Linux 的动态模块与设备驱动

3.1 实验目的

（1）学习内核通过动态模块来动态加载内核新功能的机制；

（2）理解 LINUX 字符设备驱动程序的基本原理；

（3）掌握字符设备的驱动运作机制；

（4）学会编写字符设备驱动程序；

（5）学会编写用户程序通过对字符设备的读写完成不同用户间的通信。

3.2 实验内容

（1）编译、安装与卸载动态模块；

（2）实现系统调用的篡改；

（3）编写一个简单的字符设备驱动程序，以内核空间模拟字符设备，完成对该设备的打开、读写和释放操作；

（4）编写聊天程序实现不同用户通过该设备的一对一、一对多、多对多聊天。

3.3 实验思想

Linux 内核采用 monolithic 架构，系统所有核心功能均在一个大的内核空间中运行。传统情况下若要扩展内核功能，需要重新配置并重新编译内核，这不仅耗时，也会影响系统的稳定性。为了解决这一问题，Linux 提供了动态可加载模块的机制，使得开发者可以在系统运行过程中动态装载、卸载内核功能，提高内核的灵活性与可扩展性。

在本实验中，通过编写和加载内核模块，掌握内核模块的构建方式、初始化与注销函数的实现、模块与内核之间的接口机制，并理解模块加载后的行为与权限。通过修改系统调用实验，体会内核中系统调用表的组织原理、特权级别、内存保护机制等。

Linux 的字符设备驱动通过 file_operations 结构体向 VFS 提供统一接口，使得用户对设备文件 open、read、write、release 等操作都能被映射到驱动程序中的对应函数。本实验将基于此机制实现一个简单的字符设备驱动，理解用户态与内核态之间的数据交互方式、主设备号与次设备号的含义、驱动的注册与注销流程。

通过编写用户态聊天程序，实现不同用户进程利用同一字符设备进行一对一、一对多、多对多通信，进一步理解驱动程序的同步与互斥逻辑，加深对 Linux 设备驱动与内核机制的理解。

3.4 实验步骤

（1）编写动态模块源码，包括模块初始化函数 mymodule_init() 与模块退出函数 mymodule_exit()，并添加 MODULE_LICENSE、MODULE_AUTHOR 等信息。

（2）编写 Makefile 文件，配置当前内核版本对应的内核源码路径，采用 obj-m 方式生成外部模块。

（3）运行 make 对模块进行编译，确认生成.ko 模块文件。

（4）加载模块：使用 insmod 将模块加载入内核，并使用 lsmod、dmesg 验证模块是否成功加载。

（5）编写系统调用篡改模块：查找系统调用表地址、保存旧系统调用函数地址、替换为自定义函数，实现 syscall 的动态替换。

（6）加载系统调用修改模块并运行测试程序，观察系统调用结果的变化，理解系统调用表修改的作用与风险。

（7）在测试程序中调用 syscall，比较修改前后系统调用返回值与行为，验证模块功能。

（8）卸载修改系统调用模块，确保系统调用表恢复正常。

（9）编写字符设备驱动程序源码：定义并实现设备的 open、read、write、release 函数；通过 register_chrdev() 注册字符设备。

（10）实现 file_operations 结构体并绑定驱动入口函数，使得读写设备文件能够触发驱动程序逻辑。

（11）加载字符设备驱动模块，确认/dev 目录下对应设备文件已创建。

（12）编写用户态读写测试程序，调用 open、read、write 访问字符设备，验证内核态与用户态数据交互。

（13）运行测试程序，观察读写设备内容、变量更新情况，分析 copy_to_user 与 copy_from_user 函数的作用。

（14）编写聊天程序，实现不同用户进程通过字符设备通讯，模拟一对一、一对

多、多对多通信模型。

（15）添加同步与互斥机制，防止并发访问造成数据覆盖或竞争。

（16）重复加载、卸载模块，观察设备驱动在不同使用场景下的行为，体会模块安装与卸载过程中的资源管理。

3.5 动态模块加载与系统调用篡改

3.5.1 进行系统调用篡改的动态模块

由于实验指导书中给出的代码示例已不适用于现代操作系统内核（新的内核版本默认不支持、也不鼓励直接篡改系统调用表），本实验在查阅相关资料的基础上，对原有方案进行了调整：通过修改 x86_64 架构下 CR0 寄存器中的 WP（Write Protect）写保护位，临时关闭内核对只读页的写保护，从而在内核态模拟实现对系统调用表 sys_call_table 的篡改。

具体而言，在基于 Linux 5.10 的 openEuler 环境中，首先利用以下指令查询到当前内核中 sys_call_table 的实际地址 0xffffffff9c0002c0：

```
cat /proc/kallsyms | awk '$3=="sys_call_table"{print $1}'
ffffffff9c0002c0
```

以硬编码的方式将该地址保存在模块源码中的 sys_call_table_addr 变量中。随后在模块中定义要替换的系统调用号 SYS_HOOK_NO 为 96，对应内核中的 SYS_gettimeofday，并通过保存原始系统调用函数指针 old_syscall_func 和修改 sys_call_table[SYS_HOOK_NO] 的方式，实现对该系统调用行为的动态替换。

在代码实现上，该模块首先必要的内核头文件，并使用 MODULE_LICENSE、MODULE_AUTHOR、MODULE_DESCRIPTION 等宏对模块元信息进行声明，保证模块可以在高版本内核中正常编译和加载。通过 module_init(sys_hook_init)、module_exit(sys_hook_exit) 与内核模块框架完成注册，保证模块可以被内核加载与卸载。

```
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Name");
MODULE_DESCRIPTION("Experiment: modify a syscall on Linux 5.10");
/* 模块加载入口：完成系统调用表篡改 */
static int __init sys_hook_init(void)
```

```
{
    pr_info("[sys_hook] module loading, trying to hook syscall %d\n",
            SYS_HOOK_NO);
    return hook_syscall();
}

/* 模块卸载入口：恢复系统调用表 */
static void __exit sys_hook_exit(void)
{
    pr_info("[sys_hook] module unloading, restoring syscall %d\n",
            SYS_HOOK_NO);
    unhook_syscall();
}

module_init(sys_hook_init);
module_exit(sys_hook_exit);
```

为突破内核只读保护，模块中实现了 `clear_wp()` 与 `restore_wp()` 两个静态函数：前者通过内联汇编指令读取 CR0 寄存器值，清除其中的 WP 位，并写回修改后的值，从而在极短时间内允许对内核只读区域进行写操作；后者则在完成篡改后恢复原始 CR0 值，重新启用写保护，避免对内核安全性造成长时间影响。

```
/*
 * 清除 CR0 中的 WP 位，使得内核代码页临时可写。
 * 返回修改前的 cr0 值，方便之后恢复。
 */
static unsigned long clear_wp(void)
{
    unsigned long cr0;
    asm volatile("mov %%cr0, %0" : "=r"(cr0));
    asm volatile("mov %0, %%cr0" :: "r"(cr0 & ~CR0_WP_BIT));
    return cr0;
}

/* 恢复 CR0 原来的值，重新开启写保护 */
static void restore_wp(unsigned long cr0)
{
    asm volatile("mov %0, %%cr0" :: "r"(cr0));
}
```

系统调用替换函数 sys_hello 使用 asmlinkage long sys_hello(const struct pt_regs *regs) 的形式定义，符合 Linux 5.10 体系结构中通过 pt_regs 传递系统调用参数的规范，通过 regs->di、regs->si 读取用户空间传入的两个长整型参数，计算它们的和并返回，同时利用 printk 输出调试信息，记录被 hook 的系统调用号及传入参数。

```
static asmlinkage long sys_hello(const struct pt_regs *regs)
{
    long a = regs->di;
    long b = regs->si;
    long r = a + b;
    printk(KERN_INFO "[sys_hello] syscall %d hooked: a=%ld, b=%ld,
result=%ld\n",
           SYS_HOOK_NO, a, b, r);
    return r;
}
```

在系统调用表实际篡改过程中，核心逻辑封装在 hook_syscall() 与 unhook_syscall() 两个函数中。hook_syscall() 首先检查 sys_call_table_addr 是否为非零，以避免未正确获取系统调用表地址时对内核进行非法写操作；随后将 sys_call_table_addr + SYS_HOOK_NO * sizeof(void *) 处强制转换为 unsigned long *，得到系统调用表中第 96 项的入口地址，并将其原始值保存到全局变量 old_syscall_func 中，便于模块卸载时恢复。接着调用 clear_wp() 临时关闭 CR0 中的 WP 写保护位，将该表项指向自定义的 sys_hello 函数指针，最后调用 restore_wp() 恢复 CR0 寄存器，完成一次原子化的“关闭保护－修改－重新保护”过程。unhook_syscall() 的实现与之相反，在模块卸载时重新计算第 96 项的表项地址，关闭写保护后将其写回为 old_syscall_func，再恢复 CR0，保证系统调用表不被永久篡改。模块入口函数 sys_hook_init() 在模块加载时调用 hook_syscall() 完成系统调用替换，出口函数 sys_hook_exit() 在模块卸载时调用 unhook_syscall() 负责恢复。

```
/*
 * 修改 sys_call_table，将 sys_call_table[SYS_HOOK_NO] 替换为 sys_hello。
 */
static int hook_syscall(void)
{
```

```
    unsigned long *syscall_entry;
    unsigned long old_cr0;

    if (!sys_call_table_addr) {
        pr_err("[sys_hook] sys_call_table_addr is 0 — please set it
correctly.\n");
        return -EINVAL;
    }

    /* 找到 sys_call_table 中第 SYS_HOOK_NO 项 */
    syscall_entry = (unsigned long *)(sys_call_table_addr +
                                      SYS_HOOK_NO * sizeof(void *));
    /* 保存原来的函数指针 */
    old_syscall_func = *syscall_entry;

    /* 临时关闭写保护，写入新的函数地址 */
    old_cr0 = clear_wp();
    *syscall_entry = (unsigned long)sys_hello;
    restore_wp(old_cr0);

    pr_info("[sys_hook] replaced syscall %d at %p, old=0x%lx, new=%p\n",
            SYS_HOOK_NO, syscall_entry, old_syscall_func, sys_hello);
    return 0;
}

/*
 * 恢复原来的系统调用。
 * 在模块卸载时调用，保证内核不会一直处于被篡改状态。
 */
static void unhook_syscall(void)
{
    unsigned long *syscall_entry;
    unsigned long old_cr0;

    /* 如果没成功 hook 过，直接返回 */
    if (!sys_call_table_addr || !old_syscall_func)
        return;
```

```
    syscall_entry = (unsigned long *)(sys_call_table_addr +
                                      SYS_HOOK_NO * sizeof(void *));
    old_cr0 = clear_wp();
    *syscall_entry = old_syscall_func;
    restore_wp(old_cr0);

    pr_info("[sys_hook] restored syscall %d to 0x%lx\n",
            SYS_HOOK_NO, old_syscall_func);
}
```

通过上述步骤，本实验成功完成了一个可在运行时加载、能够动态修改 SYS_gettimeofday 行为的内核动态模块，为后续测试程序对篡改效果的验证奠定了基础。

3.5.2 测试程序与运行结果

为验证系统调用是否被成功替换，我分别编写了 modify_old_syscall.c 与 modify_new_syscall.c 两个测试程序：前者在未加载动态模块或卸载模块之后，通过 SYS_gettimeofday 获取并打印当前时间，用于验证系统调用处于正常状态；后者在加载篡改模块之后，同样通过 SYS_gettimeofday 这一系统调用号传入两个整数参数 10 和 20，观察返回值是否变为两数之和，从而判断系统调用表是否已正确被新的 sys_hello 替换。

```
// modify_old_syscall.c
#include <stdio.h>

#include <sys/time.h>
#include <unistd.h>
#include <sys/syscall.h>

int main(void)
{
    struct timeval tv;
    long ret = syscall(SYS_gettimeofday, &tv, NULL);
    if (ret == -1) {
        perror("syscall");
        return 1;
    }
```

```
    printf("tv_sec:%ld\n", (long)tv.tv_sec);
    printf("tv_usec:%ld\n", (long)tv.tv_usec);
    return 0;
}


// modify_new_syscall.c
#include <stdio.h>

#include <unistd.h>
#include <sys/syscall.h>

int main(void)
{
    long ret = syscall(SYS_gettimeofday, 10, 20);
    printf("%ld\n", ret);
    return 0;
}
```

首先对两个用户态测试程序以及内核模块进行编译，生成可执行文件 old、new 和模块文件 modify_syscall.ko。在未加载内核模块的情况下运行 old 程序，可以看到 SYS_gettimeofday 按照原有语义正常返回当前时间，说明此时系统调用表尚未被修改；随后加载编写好的动态模块，并运行 new 程序，通过相同的系统调用号传入两个整型参数，观察返回值是否为两数之和。可以看到 ./new 的输出结果为 30，与预期的 10 + 20 一致，表明用户态调用的仍是系统调用号 96，但其内核侧实现已被替换为自定义的 sys_hello。结果如下：

```
[root@openeuler22 lab3]# gcc ./modify_old_syscall.c -o old
[root@openeuler22 lab3]# gcc ./modify_new_syscall.c -o new
[root@openeuler22 lab3]# make
make -C /lib/modules/5.10.0-60.139.0.166.oe2203.x86_64/build M=/root/lab3 modules
make[1]: Entering directory '/usr/src/kernels/5.10.0-60.139.0.166.oe2203.x86_64'
make[1]: Leaving directory '/usr/src/kernels/5.10.0-60.139.0.166.oe2203.x86_64'
[root@openeuler22 lab3]# ./old
tv_sec:1764314562
tv_usec:206274
[root@openeuler22 lab3]# insmod ./modify_syscall.ko
[root@openeuler22 lab3]# ./new
30
[root@openeuler22 lab3]# rmmod modify_syscall
```

为进一步验证内核行为，使用 dmesg | tail 查看模块加载、系统调用执行

与模块卸载过程中的内核日志：

```
[15813.758840] [sys_hook] module loading, trying to hook syscall 96
[15813.759623] [sys_hook] replaced syscall 96 at 000000003b2a56eb, old=0xffffffff9b17e9c0, new=00000000de2d888d
[15817.130992] [sys_hello] syscall 96 hooked: a=10, b=20, result=30
[15824.108691] [sys_hook] module unloading, restoring syscall 96
[15824.109469] [sys_hook] restored syscall 96 to 0xffffffff9b17e9c0
```

从日志可以清晰地看到模块加载时成功替换了系统调用号 96，对应的新函数在接收到参数 a=10、b=20 时计算结果为 30，卸载模块时又将该系统调用恢复为原始地址。

综合用户态输出与内核日志信息，可以得出结论：本实验实现的动态模块成功完成了对 SYS_gettimeofday 系统调用的篡改，并在模块卸载后正确恢复了系统调用表。

3.5.3 小结与安全性分析

本节通过编写并加载内核动态模块，在 Linux 5.10 环境下成功实现了对系统调用表的在线篡改，将 SYS_gettimeofday 替换为自定义的 sys_hello，并通过两个用户态测试程序验证了篡改前后系统调用行为的变化，证明模块实现正确。需要注意的是，本实验采用硬编码的方式获取 sys_call_table 地址，并通过修改 CR0 寄存器关闭写保护来完成表项替换，这种方式仅适用于当前内核版本，且会暂时削弱内核自身的保护机制，在实际生产环境中并不推荐使用。本实验主要用于加深对内核模块机制、系统调用实现方式以及内核地址保护机制的理解。

3.6 Linux 设备驱动

3.6.1 字符设备驱动程序的实现

本实验基于字符设备驱动实现了一个内核态“聊天室”，主设备号为 CHAT_MAJOR = 290，设备名为 "chatdev"，通过 register_chrdev(CHAT_MAJOR, "chatdev", &chat_fops) 完成注册。驱动内部定义了一个全局的聊天设备结构体 struct chat_dev_t，包含一个大小为 4KB 的环形缓冲区 buf[BUF_SIZE]、全局写指针 head、互斥锁 mutex 以及读进程等待队列 wait_queue_head_t wq，用于在多用户环境下缓存消息并协调并发访问。

```
struct chat_dev_t {
    char buf[BUF_SIZE];
```

```
    unsigned long head;       // Global write pointer
    struct mutex lock;        // Protects buf and head
    wait_queue_head_t wq;     // Wait queue for readers
};

static struct chat_dev_t chat_dev;
```

为了让每个打开设备的进程拥有独立的“阅读进度”，驱动为每个 file 维护一个私有数据 struct chat_private，其中仅包含一个读指针 tail，表示该进程当前读到的缓冲区位置。chat_open 中通过 kmalloc 分配 struct chat_private，并在持有 chat_dev.lock 的情况下，根据当前全局 head 初始化 tail：若历史消息超过 4KB，则只从缓冲区中最旧的可见位置开始读取，否则从 0 开始。chat_release 中释放该私有结构体内存。

```
// Per-file private data
struct chat_private {
    unsigned long tail;       // Local read pointer
};

static int chat_open(struct inode *inode, struct file *filp)
{
    struct chat_private *priv;

    priv = kmalloc(sizeof(struct chat_private), GFP_KERNEL);
    if (!priv)
        return -ENOMEM;

    mutex_lock(&chat_dev.lock);
    // New user starts reading from the oldest available data (history)
    if (chat_dev.head > BUF_SIZE)
        priv->tail = chat_dev.head - BUF_SIZE;
    else
        priv->tail = 0;
    mutex_unlock(&chat_dev.lock);

    filp->private_data = priv;
    return 0;
}
```

```
static int chat_release(struct inode *inode, struct file *filp)
{
    kfree(filp->private_data);
    return 0;
}
```

读操作 chat_read 采用阻塞方式实现：首先使用 wait_event_interruptible(chat_dev.wq, chat_dev.head != priv->tail) 等待缓冲区中出现新数据，当其它进程写入新消息并唤醒等待队列后，当前读进程获得唤醒机会。进入临界区后，通过比较 chat_dev.head - priv->tail 检查是否出现“读者太慢导致被覆盖”的情况，若差值大于缓冲区大小，则将 tail 前移到 head - BUF_SIZE 位置，只保留最近 4KB 的最新消息。随后根据用户请求长度 len 和可读字节数计算本次读取的实际字节数 count，并通过 (priv->tail + i) & (BUF_SIZE - 1) 的方式在环形缓冲区中取数据，利用 put_user 拷贝到用户空间，最后更新 priv->tail。

```
static ssize_t chat_read(struct file *filp, char __user *buf,
                         size_t len, loff_t *off)
{
    struct chat_private *priv = filp->private_data;
    int ret = 0;
    unsigned long count;
    int i;

    // Wait until there is new data (head > tail)
    if (wait_event_interruptible(chat_dev.wq, chat_dev.head !=
priv->tail))
        return -ERESTARTSYS;

    mutex_lock(&chat_dev.lock);

    // Check for buffer overflow (reader too slow)
    if (chat_dev.head - priv->tail > BUF_SIZE) {
        priv->tail = chat_dev.head - BUF_SIZE;
    }

    count = chat_dev.head - priv->tail;
```

```
    if (count > len)
        count = len;

    // Copy data to user
    for (i = 0; i < count; i++) {
        char c = chat_dev.buf[(priv->tail + i) & (BUF_SIZE - 1)]; //
Circular buffer
        if (put_user(c, buf + i)) {
            // Failed when return a non-zero value
            ret = -EFAULT;
            goto out;
        }
    }

    priv->tail += count;
    ret = count;

out:
    mutex_unlock(&chat_dev.lock);
    return ret;
}
```

写操作 chat_write 负责将用户发送的聊天消息写入环形缓冲区。函数首先限制单次写入长度不能超过缓冲区大小（len > BUF_SIZE 时直接返回 -EINVAL），然后在持有互斥锁的情况下，利用 get_user 从用户空间逐字节取出数据写入 chat_dev.buf[(chat_dev.head + i) & (BUF_SIZE - 1)]，并将 head 前移 len 个字节。写入完成后通过 wake_up_interruptible(&chat_dev.wq) 唤醒所有在等待队列上阻塞的读进程，实现消息广播。

```
static ssize_t chat_write(struct file *filp, const char __user *buf,
                          size_t len, loff_t *off)
{
    int ret = 0;
    int i;

    if (len > BUF_SIZE)
        return -EINVAL; // Message too large
```

```
    mutex_lock(&chat_dev.lock);

    for (i = 0; i < len; i++) {
        char c;
        if (get_user(c, buf + i)) {
            ret = -EFAULT;
            goto out;
        }
        chat_dev.buf[(chat_dev.head + i) & (BUF_SIZE - 1)] = c;
    }

    chat_dev.head += len;
    ret = len;

    // Wake up all readers
    wake_up_interruptible(&chat_dev.wq);

out:
    mutex_unlock(&chat_dev.lock);
    return ret;
}
```

此外，驱动还实现了 chat_poll 接口以支持 select/poll 等多路复用机制，在用户程序使用 select() 监听设备时，当 chat_dev.head != priv->tail（即有新数据可读）时返回 EPOLLIN | EPOLLRDNORM，表明设备处于可读状态。模块初始化函数 chat_init 负责初始化互斥锁、等待队列和 head 指针，并完成字符设备注册，退出函数 chat_exit 则在卸载模块时注销设备。

```
static __poll_t chat_poll(struct file *filp, poll_table *wait)
{
    struct chat_private *priv = filp->private_data;
    __poll_t mask = 0;

    poll_wait(filp, &chat_dev.wq, wait);

    mutex_lock(&chat_dev.lock);
    if (chat_dev.head != priv->tail)
        mask |= EPOLLIN | EPOLLRDNORM;
```

```
    mutex_unlock(&chat_dev.lock);

    return mask;
}

static struct file_operations chat_fops = {
    .owner   = THIS_MODULE,
    .read    = chat_read,
    .write   = chat_write,
    .poll    = chat_poll,
    .open    = chat_open,
    .release = chat_release,
};

static int __init chat_init(void)
{
    int ret;

    // Initialize device structure
    mutex_init(&chat_dev.lock);
    init_waitqueue_head(&chat_dev.wq);
    chat_dev.head = 0;

    ret = register_chrdev(CHAT_MAJOR, "chatdev", &chat_fops);
    if (ret < 0) {
        printk(KERN_ERR "chatdev: register_chrdev failed\n");
        return ret;
    }

    printk(KERN_INFO "chatdev: module loaded\n");
    return 0;
}

static void __exit chat_exit(void)
{
    unregister_chrdev(CHAT_MAJOR, "chatdev");
    printk(KERN_INFO "chatdev: module unloaded\n");
}
```

```
module_init(chat_init);
module_exit(chat_exit);
```

3.6.2 用户态聊天室程序的实现

用户态程序通过打开字符设备 /dev/chatdev 与内核聊天设备交互，每个进程对应一个聊天室用户。程序启动时从命令行参数中读取用户名，并以只读写打开设备文件：

```
#define DEVICE_PATH "/dev/chatdev"

if (argc < 2) {
    printf("Usage: %s <username>\n", argv[0]);
    return 1;
}
strncpy(name, argv[1], 31);
name[31] = '\0';

fd = open(DEVICE_PATH, O_RDWR);
if (fd < 0) {
    perror("Failed to open device");
    return 1;
}
```

主循环中使用 select() 同时监听标准输入（STDIN_FILENO）和设备文件描述符 fd，实现“边收消息边发消息”的功能。当标准输入就绪时，通过 fgets 读取用户输入，并去掉末尾换行符。程序约定两种发送模式：普通消息直接广播，格式为 "[用户名] 消息内容\n"，私聊消息使用命令 /w <user> <message>，格式为 "[发送者->接收者] 消息内容\n"。对应代码如下：

```
FD_ZERO(&fds);
FD_SET(STDIN_FILENO, &fds);
FD_SET(fd, &fds);

max_fd = (fd > STDIN_FILENO) ? fd : STDIN_FILENO;

int ret = select(max_fd + 1, &fds, NULL, NULL, NULL);
if (ret < 0) {
```

```
    if (errno == EINTR) continue;
    perror("select");
    break;
}

// Check for user input
if (FD_ISSET(STDIN_FILENO, &fds)) {
    if (fgets(write_buf, BUF_SIZE, stdin) == NULL) {
        break; // EOF
    }
    // Remove newline
    write_buf[strcspn(write_buf, "\n")] = 0;
    if (strlen(write_buf) > 0) {
        if (strncmp(write_buf, "/w ", 3) == 0) {
            char target[32], content[BUF_SIZE];
            if (sscanf(write_buf + 3, "%31s %255[^\n]", target, content)
== 2) {
                snprintf(msg_buf, sizeof(msg_buf), "[%s->%s] %s\n",
name, target, content);
                write(fd, msg_buf, strlen(msg_buf));
            } else {
                printf("Usage: /w <user> <message>\n");
            }
        } else {
            snprintf(msg_buf, sizeof(msg_buf), "[%s] %s\n", name,
write_buf);
            write(fd, msg_buf, strlen(msg_buf));
        }
    }
}
```

当设备文件描述符就绪时，程序调用 `read(fd, read_buf, sizeof(read_buf) - 1)` 从驱动中取出新消息，并根据消息格式进行分类显示。对于普通广播消息，仍然使用前缀过滤的方式尽量不重复打印自己的消息：构造前缀字符串 `"[用户名]"`，如果读出的内容以该前缀开头，则认为是该用户自己刚刚发送的广播消息，可以选择不重复显示。而对于私聊消息，程序检测是否包含 `"->"`，并进一步判断当前用户是否是发送者或接收者：如果当前用户是“私聊发送

者”或“私聊接收者”，则显示该条私聊消息；如果当前用户既不是发送者也不是接收者，则不显示这条私聊消息，从而在“显示层”上实现一对一私聊的效果。

核心代码如下：

```
// Check for device output
if (FD_ISSET(fd, &fds)) {
    int n = read(fd, read_buf, sizeof(read_buf) - 1);
    if (n > 0) {
        read_buf[n] = '\0';
        // Check for private message marker
        if (strstr(read_buf, "->")) {
            char my_target[64];
            char my_sender[64];
            snprintf(my_target, sizeof(my_target), "->%s]", name);
            snprintf(my_sender, sizeof(my_sender), "[%s->", name);
            // Show if I am the target OR I am the sender
            if (strstr(read_buf, my_target) || strstr(read_buf,
my_sender)) {
                  printf("%s", read_buf);
                  fflush(stdout);
            }
       } else {
            // Broadcast message
            char my_prefix[64];
            snprintf(my_prefix, sizeof(my_prefix), "[%s]", name);
            // Don't echo my own broadcast messages
            if (strstr(read_buf, my_prefix) != read_buf) {
                printf("%s", read_buf);
                fflush(stdout);
            }
        }
    } else if (n < 0) {
        perror("read");
        break;
    }
}
```

通过上述设计，任意多个用户态进程运行该程序并同时打开/dev/chatdev，即可在同一聊天室中共享消息流，实现简易的多人聊天与私聊功能。

3.6.3 读写同步与互斥机制

在内核驱动内部，对共享数据结构 chat_dev.buf 与 chat_dev.head 的访问通过 mutex 进行互斥保护，保证任意时刻只有一个执行线程（读或写）可以修改这些共享变量，避免了并发条件下的竞态问题。所有与缓冲区相关的读写操作（包括更新 head、更新各个进程的 tail）都在 mutex_lock(&chat_dev.lock) 与 mutex_unlock(&chat_dev.lock) 之间完成，从而实现多用户并发场景下的互斥访问。

同步方面，驱动采用等待队列 wait_queue_head_t wq 配合 wait_event_interruptible() 和 wake_up_interruptible() 实现“写消息唤醒读者”的机制：当读进程在调用 chat_read 时发现当前没有新数据（head == priv->tail），会在等待队列上睡眠，直到其他用户写入新消息并调用 wake_up_interruptible(&chat_dev.wq) 才会被唤醒，从而实现了阻塞式读取和多用户消息推送。在用户态，程序使用 select() 同时监听终端输入和设备可读状态，避免了忙轮询，提高了整个聊天室的交互性和资源利用率。

3.6.4 一对一、一对多、多对多聊天的实现方式

本实验的聊天模型本质上是基于字符设备的广播式消息通道：所有写入 /dev/chatdev 的消息都会被追加到驱动维护的全局环形缓冲区中，而每个打开设备的进程都维护自己独立的读指针 tail，从而可以按照自己的节奏从同一缓冲区中依次读取所有消息。这种设计天然支持多种聊天模式：

- 一对一聊天：当只有两个用户进程同时打开设备并互相发送消息时，设备中的所有消息仅在这两人之间传递，对这两个人来说就是一对一聊天的场景；当聊天室中有多人时，以“/w <user> <message>”的格式发送消息，即可实现与某用户的私聊，其他用户不可见。
- 一对多聊天：当一个用户发送消息，多个用户只读不写时，发送者的每条消息都会被写入全局缓冲区，并被所有接收者进程从各自的 tail 位置读出，相当于一个发送者广播给多个接收者；
- 多对多聊天：当多个用户同时读写设备时，每个人发送的消息都会追加到同一缓冲区，并被所有在线用户读取，形成了典型的多人聊天室场景。

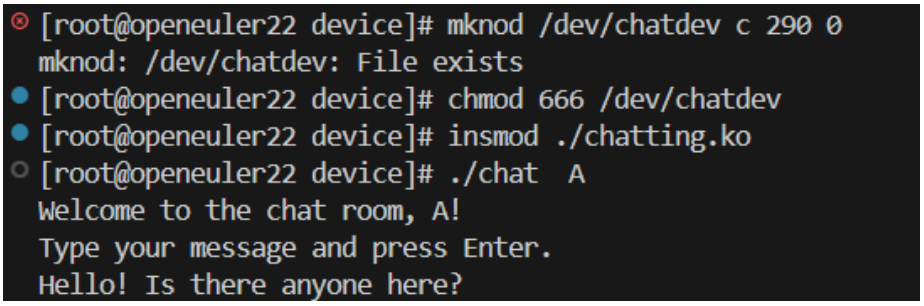
驱动层并不区分“发送给谁”，而是对所有打开设备的进程一视同仁进行广播；聊天的“身份”和“来源”通过用户态程序在消息前加上的 "[用户名]" 前缀进

行区分。不同聊天模式的本质差别主要体现在参与进程的数量和读写角色上，而驱动本身始终提供统一的多对多广播机制，从而满足实验中对“一对一、一对多、多对多聊天”的要求。

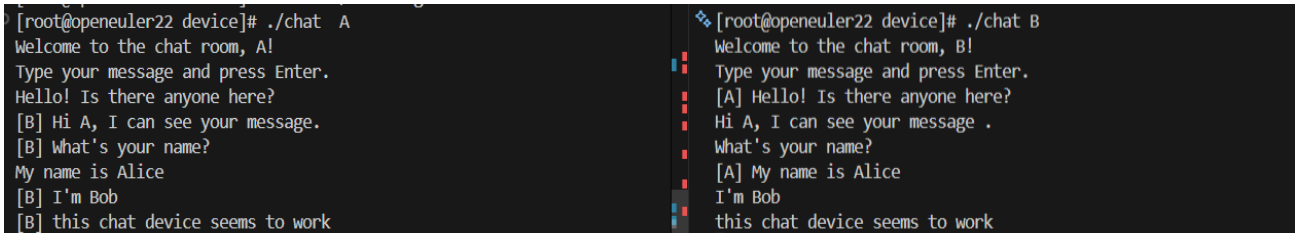
3.6.4 实验结果

在加载聊天字符设备驱动模块后，通过 mknod /dev/chatdev c 290 0 手动创建设备节点，将主设备号 290、次设备号 0 的字符设备映射为 /dev 目录中的一个特殊文件；随后使用 chmod 666 /dev/chatdev 赋予所有用户对该设备的读写权限，以便多个普通用户进程可以同时通过 /dev/chatdev 参与聊天室通信。

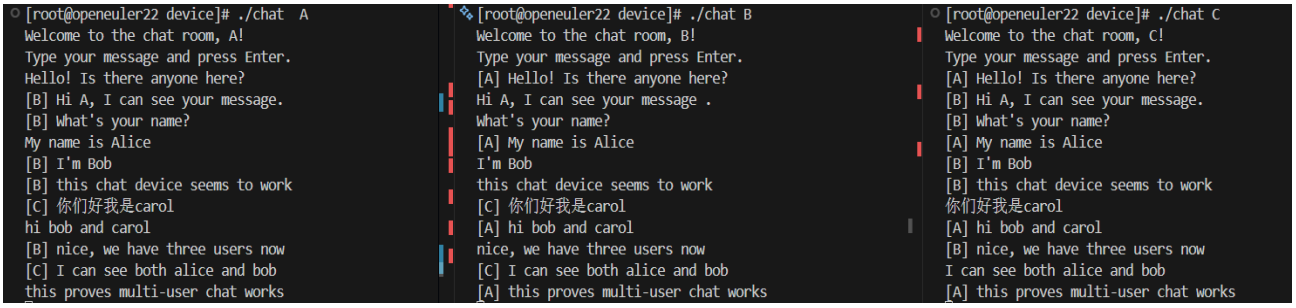
首先，用户 A 加入聊天室并发送了一条消息，此时聊天室中只有一个用户。



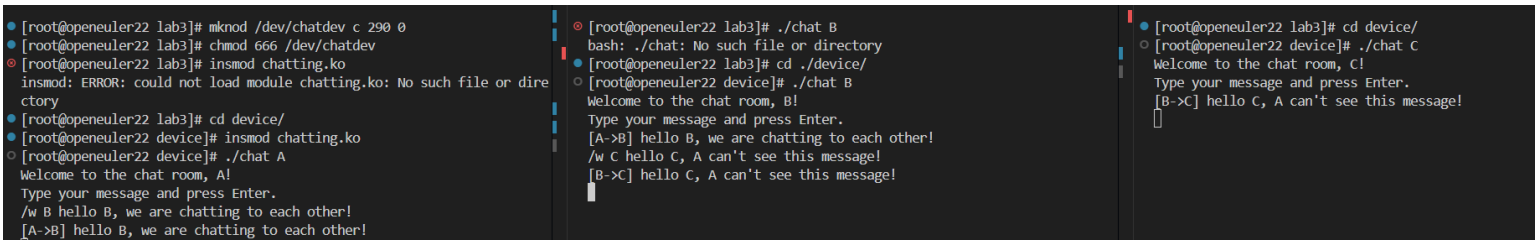
随后，用户 B 加入聊天室，可以观察到 B 能够收到 A 发送的历史消息。随后，B 与 A 进行聊天，可以观察到双方的消息读写都正常运行。



之后，用户 C 加入，进行多人聊天：



A 与 B、B 与 C 进行私聊，测试私聊功能：



三个用户都可以正常通过预设的设备文件进行聊天，符合预期的聊天室功能。

3.7 实验总结

3.7.1 实验中的问题与解决过程

1、指导书示例代码与现代内核不兼容：刚开始照着实验指导书的示例写系统调用篡改模块，发现在 Linux 5.10 环境下发现无法直接修改 sys_call_table，示例中的接口和写法也已经过时。之后通过查资料，采用了“查出 sys_call_table 实际地址 + 修改 CR0 中 WP 写保护位”的方式，重新设计了模块结构，经过多次加载、卸载模块和测试程序验证，最终在 5.10 内核上正确实现了 SYS_gettimeofday 的替换。

2、字符设备驱动的状态管理一度设计不当：在实现聊天室驱动时，一开始将读写指针都写成全局变量，结果导致所有用户共享同一个“读位置”：某个客户端一旦 read() 过，其他客户端就看不到之前的消息。经过调试后，意识到应该将“每个打开实例的状态”挂在 struct file 的 private_data 上，于是引入 struct chat_private { unsigned long tail; }，在 open 时为每个 filp 分配一份，并在 read / poll 中使用各自的 tail。改完后，各个客户端可以独立从全局环形缓冲区中按自己的进度阅读消息。

3、环形缓冲区读写索引处理错误导致数据错乱：在实现环形缓冲区时，最初简单用 buf[head + i]、buf[tail + i] 访问数组，没有考虑越界和回绕问题，当消息量较多时读取出现乱码和异常。随后结合 BUF_SIZE 为 2 的幂的特点，改用 (index) & (BUF_SIZE - 1) 实现快速取模，并在 chat_read 中增加“读者太慢”的检查逻辑：当 head - tail > BUF_SIZE 时，将 tail 强制追至 head - BUF_SIZE，只保留最新的 4KB 消息，从而解决了缓冲区覆盖和访问越界问题。

4、poll_wait / select 协作机制不理解导致 select() 行为异常：在用户程序中使用

select() 同时监听标准输入和 /dev/chatdev 时，一开始在驱动的 chat_poll 中只判断 head != tail 返回 EPOLLIN，没有调用 poll_wait，结果 select() 要么一直阻塞，要么无法被正确唤醒。查阅资料后了解到：poll_wait(filp, &chat_dev.wq, wait) 的作用是将当前进程挂到等待队列上，只有这样 wake_up_interruptible(&chat_dev.wq) 才能唤醒 select()。补上 poll_wait 后，select() 才能在新消息写入时被唤醒，聊天室程序可以正常边收边发。

3.7.2 实验收获

1、理解了 Linux 动态内核模块的基本使用方法。通过编写两个内核模块（系统调用篡改模块、字符设备驱动模块），实践了动态模块的使用方法：

①模块源码编写（module_init / module_exit、MODULE_LICENSE 等）；

②编写内核模块专用 Makefile，通过内核构建系统生成 .ko；

③使用 insmod / rmmod 动态加载、卸载模块，结合 lsmod、dmesg 观察模块状态和日志。 对“内核功能可以在运行时按需装卸”的概念有了直观认识，也体会到模块代码一旦出错可能直接导致系统异常，必须更加谨慎。

2、加深了对系统调用实现与内核保护机制的理解。在实现 SYS_gettimeofday 的篡改过程中，实践了：

①如何通过 sys_call_table 将系统调用号映射到函数指针；

②CR0 中 WP 写保护位对内核只读区域的保护作用；

③通过 pt_regs 在 5.10 内核中获取系统调用参数。同时也体会到，这种硬编码地址 + 修改 sys_call_table 的方式虽然能完成实验，但在真实系统中风险很高。

3、掌握了字符设备驱动的基本框架与 file_operations 机制。在聊天室驱动部分，完整实现了一个基础的字符设备驱动：

①使用 register_chrdev 注册主设备号，提供 open、release、read、write、poll 等回调；

②利用 struct file + private_data 保存每个打开实例的状态；

③使用 copy_to_user / copy_from_user、get_user / put_user 在内核态与用户态之间安全传递数据。 通过这个过程，我对 VFS 通过 file_operations 将用户的系统调用分

发到具体驱动的机制有了更清晰的认识。

4、体验了基于字符设备的一对多、多对多通信设计。聊天室驱动将字符设备抽象为一个“共享消息通道”，所有写入 /dev/chatdev 的消息进入同一个环形缓冲区，所有打开该设备的进程都可以从自己的 tail 开始依次读取，实现了：

①只有两个客户端时的“一对一”聊天与多用户下的私聊；

②一个发送、多方只读时的“一对多”广播；

③多个客户端同时读写时的典型“多对多聊天室”。 这一设计让“用驱动实现进程间通信”的概念从抽象变成了可运行的程序。

5、熟悉了等待队列与多路复用在同步中的具体用法。实验中同时用到了：

①内核等待队列 + wait_event_interruptible / wake_up_interruptible 实现阻塞式读取与唤醒；

②驱动中的 poll 回调 + 用户态的 select 协同工作，实现单线程同时等待键盘输入和设备数据。 对于“如何在内核中等待事件、如何在用户态优雅地同时等待多个 IO 源”的理解更加具体，也为后续理解 epoll、网络编程等打下基础。

3.7.3 意见与建议

1、建议更新实验指导书中的示例代码。本次在参考实验指导书示例代码实现系统调用篡改时，遇到了较多“版本不兼容”的问题，例如直接修改系统调用表的方式在 5.10 内核下已经不再适用，部分接口和写法也已过时。建议在后续版本中增加基于现代内核版本的参考代码或伪代码，以及如何获取 sys_call_table 地址、如何处理 CR0 的写保护等方法。这样可以减少同学在环境差异上的时间消耗，把精力更多放在理解原理上。

2、可以在实验文档中增加对关键结构与机制的简要说明。在写驱动时，struct file、private_data、wait_queue_head_t、poll_wait 等结构和函数需要自己查内核文档和示例才能理解。建议在实验指导书中简要介绍：file_operations、struct file、private_data 等，并解释常见内核错误码（如 -EFAULT、-ERESTARTSYS、-EINVAL）的典型用途。

3.8 附件

3.8.1 代码

1、modify_syscall.c

```c
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/kallsyms.h>
#include <linux/kprobes.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Name");
MODULE_DESCRIPTION("Experiment: modify a syscall on Linux 5.10");

/* 要篡改的系统调用号：SYS_gettimeofday */
#define SYS_HOOK_NO 96

/* CR0 中的 WP 写保护位，用来临时关闭内核只读页保护 */
#define CR0_WP_BIT 0x00010000UL

/* 记录原来的系统调用函数指针 */
static unsigned long old_syscall_func = 0;

/*
 * sys_call_table 在当前内核中的地址
 */
static unsigned long sys_call_table_addr = 0xffffffff9c0002c0;

/*
 * 清除 CR0 中的 WP 位，使得内核代码页临时可写。
 * 返回修改前的 cr0 值，方便之后恢复。
 */
static unsigned long clear_wp(void)
{
    unsigned long cr0;
    asm volatile("mov %%cr0, %0" : "=r"(cr0));
    asm volatile("mov %0, %%cr0" :: "r"(cr0 & ~CR0_WP_BIT));
    return cr0;
```

```
}

/* 恢复 CR0 原来的值，重新开启写保护 */
static void restore_wp(unsigned long cr0)
{
    asm volatile("mov %0, %%cr0" :: "r"(cr0));
}

/*
 * “伪系统调用”实现：接受两个整数参数，返回它们的和。
 */
static asmlinkage long sys_hello(const struct pt_regs *regs)
{
    long a = regs->di;
    long b = regs->si;
    long r = a + b;
    printk(KERN_INFO "[sys_hello] syscall %d hooked: a=%ld, b=%ld,
result=%ld\n",
           SYS_HOOK_NO, a, b, r);
    return r;
}

/*
 * 修改 sys_call_table，将 sys_call_table[SYS_HOOK_NO] 替换为 sys_hello。
 */
static int hook_syscall(void)
{
    unsigned long *syscall_entry;
    unsigned long old_cr0;

    if (!sys_call_table_addr) {
        pr_err("[sys_hook] sys_call_table_addr is 0 — please set it
correctly.\n");
        return -EINVAL;
    }

    /* 找到 sys_call_table 中第 SYS_HOOK_NO 项 */
    syscall_entry = (unsigned long *)(sys_call_table_addr +
```

```
                                    SYS_HOOK_NO * sizeof(void *));
    /* 保存原来的函数指针 */
    old_syscall_func = *syscall_entry;

    /* 临时关闭写保护，写入新的函数地址 */
    old_cr0 = clear_wp();
    *syscall_entry = (unsigned long)sys_hello;
    restore_wp(old_cr0);

    pr_info("[sys_hook] replaced syscall %d at %p, old=0x%lx, new=%p\n",
            SYS_HOOK_NO, syscall_entry, old_syscall_func, sys_hello);
    return 0;
}

/*
 * 恢复原来的系统调用。
 * 在模块卸载时调用，保证内核不会一直处于被篡改状态。
 */
static void unhook_syscall(void)
{
    unsigned long *syscall_entry;
    unsigned long old_cr0;

    /* 如果没成功 hook 过，直接返回 */
    if (!sys_call_table_addr || !old_syscall_func)
        return;

    syscall_entry = (unsigned long *)(sys_call_table_addr +
                                    SYS_HOOK_NO * sizeof(void *));
    old_cr0 = clear_wp();
    *syscall_entry = old_syscall_func;
    restore_wp(old_cr0);

    pr_info("[sys_hook] restored syscall %d to 0x%lx\n",
            SYS_HOOK_NO, old_syscall_func);
}

/* 模块加载入口：完成系统调用表篡改 */
```

```
static int __init sys_hook_init(void)
{
    pr_info("[sys_hook] module loading, trying to hook syscall %d\n",
            SYS_HOOK_NO);
    return hook_syscall();
}

/* 模块卸载入口：恢复系统调用表 */
static void __exit sys_hook_exit(void)
{
    pr_info("[sys_hook] module unloading, restoring syscall %d\n",
            SYS_HOOK_NO);
    unhook_syscall();
}

module_init(sys_hook_init);
module_exit(sys_hook_exit);
```

2、modify_old_syscall.c

```
#include <stdio.h>
#include <sys/time.h>
#include <unistd.h>
#include <sys/syscall.h>

int main(void)
{
    struct timeval tv;
    long ret = syscall(SYS_gettimeofday, &tv, NULL);
    if (ret == -1) {
        perror("syscall");
        return 1;
    }

    printf("tv_sec:%ld\n", (long)tv.tv_sec);
    printf("tv_usec:%ld\n", (long)tv.tv_usec);
    return 0;
```

```
}
```

3、modify_new_syscall.c

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

int main(void)
{
    long ret = syscall(SYS_gettimeofday, 10, 20);
    printf("%ld\n", ret);
    return 0;
}
```

4、chatting.c

```c
#include <linux/module.h>
#include <linux/init.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/mutex.h>
#include <linux/wait.h>
#include <linux/slab.h>
#include <linux/sched.h>
#include <linux/poll.h>

#define CHAT_MAJOR 290
#define BUF_SIZE   4096 // 4KB buffer

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Name");
MODULE_DESCRIPTION("Multi-user Chat Room Device");

struct chat_dev_t {
    char buf[BUF_SIZE];
    unsigned long head;       // Global write pointer
    struct mutex lock;        // Protects buf and head
```

```
    wait_queue_head_t wq;     // Wait queue for readers
};

static struct chat_dev_t chat_dev;

// Per-file private data
struct chat_private {
    unsigned long tail;       // Local read pointer
};

static int chat_open(struct inode *inode, struct file *filp)
{
    struct chat_private *priv;

    priv = kmalloc(sizeof(struct chat_private), GFP_KERNEL);
    if (!priv)
        return -ENOMEM;

    mutex_lock(&chat_dev.lock);
    // New user starts reading from the oldest available data (history)
    if (chat_dev.head > BUF_SIZE)
        priv->tail = chat_dev.head - BUF_SIZE;
    else
        priv->tail = 0;
    mutex_unlock(&chat_dev.lock);

    filp->private_data = priv;
    return 0;
}

static int chat_release(struct inode *inode, struct file *filp)
{
    kfree(filp->private_data);
    return 0;
}

static ssize_t chat_read(struct file *filp, char __user *buf,
                         size_t len, loff_t *off)
```

```
{
    struct chat_private *priv = filp->private_data;
    int ret = 0;
    unsigned long count;
    int i;

    // Wait until there is new data (head > tail)
    if (wait_event_interruptible(chat_dev.wq, chat_dev.head !=
priv->tail))
        return -ERESTARTSYS;

    mutex_lock(&chat_dev.lock);

    // Check for buffer overflow (reader too slow)
    if (chat_dev.head - priv->tail > BUF_SIZE) {
        priv->tail = chat_dev.head - BUF_SIZE;
    }

    count = chat_dev.head - priv->tail;
    if (count > len)
        count = len;

    // Copy data to user
    for (i = 0; i < count; i++) {
        char c = chat_dev.buf[(priv->tail + i) & (BUF_SIZE - 1)]; //
Circular buffer
        if (put_user(c, buf + i)) {
            // Failed when return a non-zero value
            ret = -EFAULT;
            goto out;
        }
    }

    priv->tail += count;
    ret = count;

out:
    mutex_unlock(&chat_dev.lock);
```

```
    return ret;
}

static ssize_t chat_write(struct file *filp, const char __user *buf,
                          size_t len, loff_t *off)
{
    int ret = 0;
    int i;

    if (len > BUF_SIZE)
        return -EINVAL; // Message too large

    mutex_lock(&chat_dev.lock);

    for (i = 0; i < len; i++) {
        char c;
        if (get_user(c, buf + i)) {
            ret = -EFAULT;
            goto out;
        }
        chat_dev.buf[(chat_dev.head + i) & (BUF_SIZE - 1)] = c;
    }

    chat_dev.head += len;
    ret = len;

    // Wake up all readers
    wake_up_interruptible(&chat_dev.wq);

out:
    mutex_unlock(&chat_dev.lock);
    return ret;
}

static __poll_t chat_poll(struct file *filp, poll_table *wait)
{
    struct chat_private *priv = filp->private_data;
    __poll_t mask = 0;
```

```
    poll_wait(filp, &chat_dev.wq, wait);

    mutex_lock(&chat_dev.lock);
    if (chat_dev.head != priv->tail)
        mask |= EPOLLIN | EPOLLRDNORM;
    mutex_unlock(&chat_dev.lock);

    return mask;
}

static struct file_operations chat_fops = {
    .owner   = THIS_MODULE,
    .read    = chat_read,
    .write   = chat_write,
    .poll    = chat_poll,
    .open    = chat_open,
    .release = chat_release,
};

static int __init chat_init(void)
{
    int ret;

    // Initialize device structure
    mutex_init(&chat_dev.lock);
    init_waitqueue_head(&chat_dev.wq);
    chat_dev.head = 0;

    ret = register_chrdev(CHAT_MAJOR, "chatdev", &chat_fops);
    if (ret < 0) {
        printk(KERN_ERR "chatdev: register_chrdev failed\n");
        return ret;
    }

    printk(KERN_INFO "chatdev: module loaded\n");
    return 0;
}
```

```
static void __exit chat_exit(void)
{
    unregister_chrdev(CHAT_MAJOR, "chatdev");
    printk(KERN_INFO "chatdev: module unloaded\n");
}

module_init(chat_init);
module_exit(chat_exit);
```

5、chat_user.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/select.h>
#include <errno.h>

#define DEVICE_PATH "/dev/chatdev"
#define BUF_SIZE 256

int main(int argc, char *argv[]) {
    int fd;
    char name[32];
    char read_buf[BUF_SIZE];
    char write_buf[BUF_SIZE];
    char msg_buf[BUF_SIZE + 32];
    fd_set fds;
    int max_fd;

    if (argc < 2) {
        printf("Usage: %s <username>\n", argv[0]);
        return 1;
    }
    strncpy(name, argv[1], 31);
```

```
    name[31] = '\0';

    fd = open(DEVICE_PATH, O_RDWR);
    if (fd < 0) {
        perror("Failed to open device");
        return 1;
    }

    printf("Welcome to the chat room, %s!\n", name);
    printf("Type your message and press Enter.\n");

    while (1) {
        FD_ZERO(&fds);
        FD_SET(STDIN_FILENO, &fds);
        FD_SET(fd, &fds);

        max_fd = (fd > STDIN_FILENO) ? fd : STDIN_FILENO;

        int ret = select(max_fd + 1, &fds, NULL, NULL, NULL);
        if (ret < 0) {
            if (errno == EINTR) continue;
            perror("select");
            break;
        }

        // Check for user input
        if (FD_ISSET(STDIN_FILENO, &fds)) {
            if (fgets(write_buf, BUF_SIZE, stdin) == NULL) {
                break; // EOF
            }
            // Remove newline
            write_buf[strcspn(write_buf, "\n")] = 0;

            if (strlen(write_buf) > 0) {
                // Format: [Name] Message
                snprintf(msg_buf, sizeof(msg_buf), "[%s] %s\n", name,
write_buf);
                write(fd, msg_buf, strlen(msg_buf));
```

```
            }
        }

        // Check for device output
        if (FD_ISSET(fd, &fds)) {
            int n = read(fd, read_buf, sizeof(read_buf) - 1);
            if (n > 0) {
                read_buf[n] = '\0';
                char my_prefix[64];
                snprintf(my_prefix, sizeof(my_prefix), "[%s]", name);
                if (strstr(read_buf, my_prefix) != read_buf) {
                    printf("%s", read_buf);
                    fflush(stdout);
                }
            } else if (n < 0) {
                perror("read");
                break;
            }
        }
    }

    close(fd);
    return 0;
}
```

3.8.2 Readme

3 Linux 的动态模块与设备驱动

实验目的

（1）学习内核通过动态模块来动态加载内核新功能的机制；

（2）理解 LINUX 字符设备驱动程序的基本原理；

（3）掌握字符设备的驱动运作机制；

（4）学会编写字符设备驱动程序；

（5）学会编写用户程序通过对字符设备的读写完成不同用户间的通信。

实验内容

（1）编译、安装与卸载动态模块；

（2）实现系统调用的篡改；

（3）编写一个简单的字符设备驱动程序，以内核空间模拟字符设备，完成对该设备的打开、读写和释放操作；

（4）编写聊天程序实现不同用户通过该设备的一对一、一对多、多对多聊天。

实验思想

Linux 内核采用 monolithic 架构，系统所有核心功能均在一个大的内核空间中运行。传统情况下若要扩展内核功能，需要重新配置并重新编译内核，这不仅耗时，也会影响系统的稳定性。为了解决这一问题，Linux 提供了动态可加载模块的机制，使得开发者可以在系统运行过程中动态装载、卸载内核功能，提高内核的灵活性与可扩展性。

在本实验中，通过编写和加载内核模块，掌握内核模块的构建方式、初始化与注销函数的实现、模块与内核之间的接口机制，并理解模块加载后的行为与权限。通过修改系统调用实验，体会内核中系统调用表的组织原理、特权级别、内存保护机制等。

Linux 的字符设备驱动通过`file_operations`结构体向 VFS 提供统一接口，使得用户对设备文件 open、read、write、release 等操作都能被映射到驱动程序中的对应函数。

本实验将基于此机制实现一个简单的字符设备驱动，理解用户态与内核态之间的数据交互方式、主设备号与次设备号的含义、驱动的注册与注销流程。

通过编写用户态聊天程序，实现不同用户进程利用同一字符设备进行一对一、一对多、多对多通信，进一步理解驱动程序的同步与互斥逻辑，加深对 Linux 设备驱动与内核机制的理解。

实验步骤

（1）编写动态模块源码，包括模块初始化函数 `mymodule_init()` 与模块退出函数 `mymodule_exit()`，并添加 MODULE_LICENSE、MODULE_AUTHOR 等信息。

（2）编写 Makefile 文件，配置当前内核版本对应的内核源码路径，采用 obj-m 方式生成外部模块。

（3）运行`make`对模块进行编译，确认生成.ko 模块文件。

（4）加载模块：使用`insmod`将模块加载入内核，并使用`lsmod`、`dmesg`验证模块是否成功加载。

（5）编写系统调用篡改模块：查找系统调用表地址、保存旧系统调用函数地址、替换为自定义函数，实现`syscall`的动态替换。

（6）加载系统调用修改模块并运行测试程序，观察系统调用结果的变化，理解系统调用表修改的作用与风险。

（7）在测试程序中调用`syscall`，比较修改前后系统调用返回值与行为，验证模块功能。

（8）卸载修改系统调用模块，确保系统调用表恢复正常。

（9）编写字符设备驱动程序源码：定义并实现设备的 `open`、`read`、`write`、`release` 函数；通过 `register_chrdev()` 注册字符设备。

（10）实现 `file_operations` 结构体并绑定驱动入口函数，使得读写设备文件能够触发驱动程序逻辑。

（11）加载字符设备驱动模块，确认 /dev/ 目录下对应设备文件已创建。

（12）编写用户态读写测试程序，调用 `open`、`read`、`write` 访问字符设备，验证内核态与用户态数据交互。

（13）运行测试程序，观察读写设备内容、变量更新情况，分析 `copy_to_user` 与 `copy_from_user` 函数的作用。

（14）编写聊天程序，实现不同用户进程通过字符设备通讯，模拟一对一、一对多、多对多通信模型。

（15）添加同步与互斥机制，防止并发访问造成数据覆盖或竞争。

（16）重复加载、卸载模块，观察设备驱动在不同使用场景下的行为，体会模块安装与卸载过程中的资源管理。

动态模块加载与系统调用篡改

进行系统调用篡改的动态模块

由于实验指导书中给出的代码示例已不适用于现代操作系统内核（新的内核版本默

认不支持、也不鼓励直接篡改系统调用表），本实验在查阅相关资料的基础上，对原有方案进行了调整：通过修改 x86_64 架构下 CR0 寄存器中的 WP（Write Protect）写保护位，临时关闭内核对只读页的写保护，从而在内核态模拟实现对系统调用表 `sys_call_table` 的篡改。

具体而言，在基于 Linux 5.10 的 openEuler 环境中，首先利用以下指令查询当前内核中 `sys_call_table` 的实际地址`0xffffffff9c0002c0`：

```bash
cat /proc/kallsyms | awk '$3=="sys_call_table"{print $1}'
ffffffff9c0002c0
```

以硬编码的方式将该地址保存在模块源码中的 `sys_call_table_addr` 变量中。随后在模块中定义要替换的系统调用号 `SYS_HOOK_NO` 为 96，对应内核中的 `SYS_gettimeofday`，并通过保存原始系统调用函数指针 `old_syscall_func` 和修改 `sys_call_table[SYS_HOOK_NO]` 的方式，实现对该系统调用行为的动态替换。

在代码实现上，该模块首先必要的内核头文件，并使用 `MODULE_LICENSE`、`MODULE_AUTHOR`、`MODULE_DESCRIPTION` 等宏对模块元信息进行声明，保证模块可以在高版本内核中正常编译和加载。通过 `module_init(sys_hook_init)`、`module_exit(sys_hook_exit)` 与内核模块框架完成注册，保证模块可以被内核加载与卸载。

```c
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Name");
MODULE_DESCRIPTION("Experiment: modify a syscall on Linux 5.10");
/* 模块加载入口：完成系统调用表篡改 */
static int __init sys_hook_init(void)
{
```

```
    pr_info("[sys_hook] module loading, trying to hook syscall %d\n",
            SYS_HOOK_NO);
    return hook_syscall();
}

/* 模块卸载入口：恢复系统调用表 */
static void __exit sys_hook_exit(void)
{
    pr_info("[sys_hook] module unloading, restoring syscall %d\n",
            SYS_HOOK_NO);
    unhook_syscall();
}

module_init(sys_hook_init);
module_exit(sys_hook_exit);
```

为突破内核只读保护，模块中实现了 `clear_wp()` 与 `restore_wp()` 两个静态函数：前者通过内联汇编指令读取 `CR0` 寄存器值，清除其中的 WP 位，并写回修改后的值，从而在极短时间内允许对内核只读区域进行写操作；后者则在完成篡改后恢复原始 CR0 值，重新启用写保护，避免对内核安全性造成长时间影响。

```c
/*
 * 清除 CR0 中的 WP 位，使得内核代码页临时可写。
 * 返回修改前的 cr0 值，方便之后恢复。
 */
static unsigned long clear_wp(void)
{
    unsigned long cr0;
    asm volatile("mov %%cr0, %0" : "=r"(cr0));
    asm volatile("mov %0, %%cr0" :: "r"(cr0 & ~CR0_WP_BIT));
```

```
    return cr0;
}

/* 恢复 CR0 原来的值，重新开启写保护 */
static void restore_wp(unsigned long cr0)
{
    asm volatile("mov %0, %%cr0" :: "r"(cr0));
}
```

系统调用替换函数 `sys_hello` 使用 `asmlinkage long sys_hello(const struct pt_regs *regs)` 的形式定义，符合 Linux 5.x 体系结构中通过 `pt_regs` 传递系统调用参数的规范，通过 `regs->di`、`regs->si` 读取用户空间传入的两个长整型参数，计算它们的和并返回，同时利用 `printk` 输出调试信息，记录被 hook 的系统调用号及传入参数。

```c
static asmlinkage long sys_hello(const struct pt_regs *regs)
{
    long a = regs->di;
    long b = regs->si;
    long r = a + b;
    printk(KERN_INFO "[sys_hello] syscall %d hooked: a=%ld, b=%ld, result=%ld\n",
           SYS_HOOK_NO, a, b, r);
    return r;
}
```

在系统调用表实际篡改过程中，核心逻辑封装在 `hook_syscall()` 与 `unhook_syscall()` 两个函数中。`hook_syscall()` 首先检查 `sys_call_table_addr` 是否为非零，以避免未正确获取系统调用表地址时对内核进行非法写操作；随后将 `sys_call_table_addr + SYS_HOOK_NO * sizeof(void *)` 处强制转换为 `unsigned long *`，得到系统调用表中第 96 项的入口地址，并将其原始值保存到全局变量

`old_syscall_func` 中，便于模块卸载时恢复。接着调用 `clear_wp()` 临时关闭 CR0 中的 WP 写保护位，将该表项指向自定义的 `sys_hello` 函数指针，最后调用 `restore_wp()` 恢复 CR0 寄存器，完成一次原子化的“关闭保护－修改－重新保护”过程。`unhook_syscall()` 的实现与之相反，在模块卸载时重新计算第 96 项的表项地址，关闭写保护后将其写回为 `old_syscall_func`，再恢复 CR0，保证系统调用表不被永久篡改。模块入口函数 `sys_hook_init()` 在模块加载时调用 `hook_syscall()` 完成系统调用替换，出口函数 `sys_hook_exit()` 在模块卸载时调用 `unhook_syscall()` 负责恢复。

```c
/*
 * 修改 sys_call_table，将 sys_call_table[SYS_HOOK_NO] 替换为 sys_hello。
 */
static int hook_syscall(void)
{
    unsigned long *syscall_entry;
    unsigned long old_cr0;

    if (!sys_call_table_addr) {
        pr_err("[sys_hook] sys_call_table_addr is 0 — please set it correctly.\n");
        return -EINVAL;
    }

    /* 找到 sys_call_table 中第 SYS_HOOK_NO 项 */
    syscall_entry = (unsigned long *)(sys_call_table_addr +
                                      SYS_HOOK_NO * sizeof(void *));
    /* 保存原来的函数指针 */
    old_syscall_func = *syscall_entry;

    /* 临时关闭写保护，写入新的函数地址 */
    old_cr0 = clear_wp();
    *syscall_entry = (unsigned long)sys_hello;
```

```
    restore_wp(old_cr0);

    pr_info("[sys_hook] replaced syscall %d at %p, old=0x%lx, new=%p\n",
            SYS_HOOK_NO, syscall_entry, old_syscall_func, sys_hello);
    return 0;
}

/*
 * 恢复原来的系统调用。
 * 在模块卸载时调用，保证内核不会一直处于被篡改状态。
 */
static void unhook_syscall(void)
{
    unsigned long *syscall_entry;
    unsigned long old_cr0;

    /* 如果没成功 hook 过，直接返回 */
    if (!sys_call_table_addr || !old_syscall_func)
        return;

    syscall_entry = (unsigned long *)(sys_call_table_addr +
                                      SYS_HOOK_NO * sizeof(void *));
    old_cr0 = clear_wp();
    *syscall_entry = old_syscall_func;
    restore_wp(old_cr0);

    pr_info("[sys_hook] restored syscall %d to 0x%lx\n",
            SYS_HOOK_NO, old_syscall_func);
}
```

通过上述步骤，本实验成功完成了一个可在运行时加载、能够动态修改 `SYS_gettimeofday` 行为的内核动态模块，为后续测试程序对篡改效果的验证奠定了

基础。

测试程序与运行结果

为验证系统调用是否被成功替换，我分别编写了 `modify_old_syscall.c` 与 `modify_new_syscall.c` 两个测试程序：前者在未加载动态模块或卸载模块之后，通过 `SYS_gettimeofday` 获取并打印当前时间，用于验证系统调用处于正常状态；后者在加载篡改模块之后，同样通过 `SYS_gettimeofday` 这一系统调用号传入两个整数参数 `10` 和 `20`，观察返回值是否变为两数之和，从而判断系统调用表是否已正确被新的 `sys_hello` 替换。

```c
// modify_old_syscall.c
#include <stdio.h>
#include <sys/time.h>
#include <unistd.h>
#include <sys/syscall.h>

int main(void)
{
    struct timeval tv;
    long ret = syscall(SYS_gettimeofday, &tv, NULL);
    if (ret == -1) {
        perror("syscall");
        return 1;
    }

    printf("tv_sec:%ld\n", (long)tv.tv_sec);
    printf("tv_usec:%ld\n", (long)tv.tv_usec);
    return 0;
}
```

```
// modify_new_syscall.c
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

int main(void)
{
    long ret = syscall(SYS_gettimeofday, 10, 20);
    printf("%ld\n", ret); // 输出应为 30
    return 0;
}
```

```

首先对两个用户态测试程序以及内核模块进行编译，生成可执行文件 `old`、`new` 和模块文件 `modify_syscall.ko`。在未加载内核模块的情况下运行 `old` 程序，可以看到 `SYS_gettimeofday` 按照原有语义正常返回当前时间，说明此时系统调用表尚未被修改；随后加载编写好的动态模块，并运行 `new` 程序，通过相同的系统调用号传入两个整型参数，观察返回值是否为两数之和。可以看到 `./new` 的输出结果为 `30`，与预期的 `10 + 20` 一致，表明用户态调用的仍是系统调用号 96，但其内核侧实现已被替换为自定义的 `sys_hello`。结果如下：




为进一步验证内核行为，使用 `dmesg | tail` 查看模块加载、系统调用执行与模块卸载过程中的内核日志：




从日志可以清晰地看到模块加载时成功替换了系统调用号 96，对应的新函数在接收
```

到参数 `a=10`、`b=20` 时计算结果为 `30`，卸载模块时又将该系统调用恢复为原始地址。

综合用户态输出与内核日志信息，可以得出结论：本实验实现的动态模块成功完成了对 `SYS_gettimeofday` 系统调用的篡改，并在模块卸载后正确恢复了系统调用表。

小结与安全性分析

本节通过编写并加载内核动态模块，在 Linux 5.10 环境下成功实现了对系统调用表的在线篡改，将 `SYS_gettimeofday` 替换为自定义的 `sys_hello`，并通过两个用户态测试程序验证了篡改前后系统调用行为的变化，证明模块实现正确。需要注意的是，本实验采用硬编码的方式获取 `sys_call_table` 地址，并通过修改 CR0 寄存器关闭写保护来完成表项替换，这种方式仅适用于当前内核版本，且会暂时削弱内核自身的保护机制，在实际生产环境中并不推荐使用。本实验主要用于加深对内核模块机制、系统调用实现方式以及内核地址保护机制的理解。

Linux 设备驱动

字符设备驱动程序的实现

本实验基于字符设备驱动实现了一个内核态“聊天室”，主设备号为 `CHAT_MAJOR = 290`，设备名为 `"chatdev"`，通过 `register_chrdev(CHAT_MAJOR, "chatdev", &chat_fops)` 完成注册。驱动内部定义了一个全局的聊天设备结构体 `struct chat_dev_t`，包含一个大小为 4KB 的环形缓冲区 `buf[BUF_SIZE]`、全局写指针 `head`、互斥锁 `mutex` 以及读进程等待队列 `wait_queue_head_t wq`，用于在多用户环境下缓存消息并协调并发访问。

```c
struct chat_dev_t {
    char buf[BUF_SIZE];
```

```
    unsigned long head;      // 全局写指针
    struct mutex lock;       // 保护 buf 和 head
    wait_queue_head_t wq;    // 读者等待队列
};

static struct chat_dev_t chat_dev;
```

为了让每个打开设备的进程拥有独立的“阅读进度”，驱动为每个 `file` 维护一个私有数据 `struct chat_private`，其中仅包含一个读指针 `tail`，表示该进程当前读到的缓冲区位置。`chat_open` 中通过 `kmalloc` 分配 `struct chat_private`，并在持有 `chat_dev.lock` 的情况下，根据当前全局 `head` 初始化 `tail`：若历史消息超过 4KB，则只从缓冲区中最旧的可见位置开始读取，否则从 0 开始。`chat_release` 中释放该私有结构体内存。

```c
struct chat_private {
    unsigned long tail;      // 每个文件独立的读指针
};

static int chat_open(struct inode *inode, struct file *filp)
{
    struct chat_private *priv = kmalloc(sizeof(*priv), GFP_KERNEL);
    if (!priv) return -ENOMEM;

    mutex_lock(&chat_dev.lock);
    if (chat_dev.head > BUF_SIZE)
        priv->tail = chat_dev.head - BUF_SIZE;
    else
        priv->tail = 0;
    mutex_unlock(&chat_dev.lock);
```

```
    filp->private_data = priv;
    return 0;
}
```

读操作 `chat_read` 采用阻塞方式实现：首先使用 `wait_event_interruptible(chat_dev.wq, chat_dev.head != priv->tail)` 等待缓冲区中出现新数据，当其它进程写入新消息并唤醒等待队列后，当前读进程获得唤醒机会。进入临界区后，通过比较 `chat_dev.head - priv->tail` 检查是否出现“读者太慢导致被覆盖”的情况，若差值大于缓冲区大小，则将 `tail` 前移到 `head - BUF_SIZE` 位置，只保留最近 4KB 的最新消息。随后根据用户请求长度 `len` 和可读字节数计算本次读取的实际字节数 `count`，并通过 `(priv->tail + i) & (BUF_SIZE - 1)` 的方式在环形缓冲区中取数据，利用 `put_user` 拷贝到用户空间，最后更新 `priv->tail`。

```c
static ssize_t chat_read(struct file *filp, char __user *buf, size_t len, loff_t *off)
{
    struct chat_private *priv = filp->private_data;
    unsigned long count;
    int i, ret = 0;

    if (wait_event_interruptible(chat_dev.wq, chat_dev.head != priv->tail))
        return -ERESTARTSYS;

    mutex_lock(&chat_dev.lock);
    if (chat_dev.head - priv->tail > BUF_SIZE)
        priv->tail = chat_dev.head - BUF_SIZE;

    count = chat_dev.head - priv->tail;
    if (count > len) count = len;

    for (i = 0; i < count; i++) {
```

```
        char c = chat_dev.buf[(priv->tail + i) & (BUF_SIZE - 1)];
        if (put_user(c, buf + i)) { ret = -EFAULT; goto out; }
    }
    priv->tail += count;
    ret = count;
out:
    mutex_unlock(&chat_dev.lock);
    return ret;
}
```

写操作 `chat_write` 负责将用户发送的聊天消息写入环形缓冲区。函数首先限制单次写入长度不能超过缓冲区大小（`len > BUF_SIZE` 时直接返回 `-EINVAL`），然后在持有互斥锁的情况下，利用 `get_user` 从用户空间逐字节取出数据写入 `chat_dev.buf[(chat_dev.head + i) & (BUF_SIZE - 1)]`，并将 `head` 前移 `len` 个字节。写入完成后通过 `wake_up_interruptible(&chat_dev.wq)` 唤醒所有在等待队列上阻塞的读进程，实现消息广播。

```c
static ssize_t chat_write(struct file *filp, const char __user *buf, size_t len, loff_t *off)
{
    int i, ret = 0;

    if (len > BUF_SIZE) return -EINVAL;

    mutex_lock(&chat_dev.lock);
    for (i = 0; i < len; i++) {
        char c;
        if (get_user(c, buf + i)) { ret = -EFAULT; goto out; }
        chat_dev.buf[(chat_dev.head + i) & (BUF_SIZE - 1)] = c;
    }
    chat_dev.head += len;
```

```
    ret = len;
    wake_up_interruptible(&chat_dev.wq);
out:
    mutex_unlock(&chat_dev.lock);
    return ret;
}
```

此外，驱动还实现了 `chat_poll` 接口以支持 `select/poll` 等多路复用机制，在用户程序使用 `select()` 监听设备时，当 `chat_dev.head != priv->tail`（即有新数据可读）时返回 `EPOLLIN | EPOLLRDNORM`，表明设备处于可读状态。模块初始化函数 `chat_init` 负责初始化互斥锁、等待队列和 `head` 指针，并完成字符设备注册，退出函数 `chat_exit` 则在卸载模块时注销设备。

用户态聊天室程序的实现

用户态程序通过打开字符设备 `/dev/chatdev` 与内核聊天设备交互，每个进程对应一个聊天室用户。程序启动时从命令行参数中读取用户名，并以只读写打开设备文件：

```c
#define DEVICE_PATH "/dev/chatdev"

int fd = open(DEVICE_PATH, O_RDWR);
if (fd < 0) {
    perror("Failed to open device");
    return 1;
}
```

主循环中使用 `select()` 同时监听标准输入（`STDIN_FILENO`）和设备文件描述符 `fd`，实现边收消息边发消息的功能。当标准输入就绪时，通过 `fgets` 读取用户输入，并去掉末尾换行符，然后按照统一格式封装为 `[用户名] 消息内容\n` 的字符串后写入设备，实现“发送消息”：

```c
FD_ZERO(&fds);
FD_SET(STDIN_FILENO, &fds);
FD_SET(fd, &fds);
max_fd = (fd > STDIN_FILENO) ? fd : STDIN_FILENO;

int ret = select(max_fd + 1, &fds, NULL, NULL, NULL);

if (FD_ISSET(STDIN_FILENO, &fds)) {
   if (fgets(write_buf, BUF_SIZE, stdin) == NULL) break;
   write_buf[strcspn(write_buf, "\n")] = 0; // 去掉换行

   snprintf(msg_buf, sizeof(msg_buf), "[%s] %s\n", name, write_buf);
   write(fd, msg_buf, strlen(msg_buf));
}
```

当设备文件描述符就绪时，程序调用 `read(fd, read_buf, sizeof(read_buf) - 1)` 从驱动中取出新消息，并直接在终端打印。为避免简单的“回声”效果，用户程序通过前缀过滤的方式尽量不重复打印自己的消息：构造前缀字符串 `"[用户名]"`，如果读出的内容以该前缀开头，则可以选择不打印，保留给其它用户使用；本实验中采用了简单的 `strstr` 判断并输出的方式，重点在于演示多进程共享同一设备实现聊天室的基本过程。

```c
if (FD_ISSET(fd, &fds)) {
   int n = read(fd, read_buf, sizeof(read_buf) - 1);
```

```
    if (n > 0) {
        read_buf[n] = '\0';
        char my_prefix[64];
        snprintf(my_prefix, sizeof(my_prefix), "[%s]", name);

        if (strstr(read_buf, my_prefix) != read_buf) {
            printf("%s", read_buf);
            fflush(stdout);
        }
    }
}
```

通过上述设计，任意多个用户态进程运行该程序并同时打开 `/dev/chatdev`，即可在同一“聊天室”中共享消息流，实现简易的多人聊天功能。

读写同步与互斥机制

在内核驱动内部，对共享数据结构 `chat_dev.buf` 与 `chat_dev.head` 的访问通过 `mutex` 进行互斥保护，保证任意时刻只有一个执行线程（读或写）可以修改这些共享变量，避免了并发条件下的竞态问题。所有与缓冲区相关的读写操作（包括更新 head、更新各个进程的 tail）都在 `mutex_lock(&chat_dev.lock)` 与 `mutex_unlock(&chat_dev.lock)` 之间完成，从而实现多用户并发场景下的互斥访问。

同步方面，驱动采用等待队列 `wait_queue_head_t wq` 配合 `wait_event_interruptible()` 和 `wake_up_interruptible()` 实现“写消息唤醒读者”的机制：当读进程在调用 `chat_read` 时发现当前没有新数据（`head == priv->tail`），会在等待队列上睡眠，直到其他用户写入新消息并调用 `wake_up_interruptible(&chat_dev.wq)` 才会被唤醒，从而实现了阻塞式读取和多用户

消息推送。在用户态，程序使用 `select()` 同时监听终端输入和设备可读状态，避免了忙轮询，提高了整个聊天室的交互性和资源利用率。

一对一、一对多、多对多聊天的实现方式

本实验的聊天模型本质上是**基于字符设备的广播式消息通道**：所有写入 `/dev/chatdev` 的消息都会被追加到驱动维护的全局环形缓冲区中，而每个打开设备的进程都维护自己独立的读指针 `tail`，从而可以按照自己的节奏从同一缓冲区中依次读取所有消息。这种设计天然支持多种聊天模式：

- **一对一聊天**：当只有两个用户进程同时打开设备并互相发送消息时，设备中的所有消息仅在这两人之间传递，对这两个人来说就是一对一聊天的场景；
- **一对多聊天**：当一个用户发送消息，多个用户只读不写时，发送者的每条消息都会被写入全局缓冲区，并被所有接收者进程从各自的 `tail` 位置读出，相当于一个发送者广播给多个接收者；
- **多对多聊天**：当多个用户同时读写设备时，每个人发送的消息都会追加到同一缓冲区，并被所有在线用户读取，形成了典型的多人聊天室场景。

驱动层并不区分“发送给谁”，而是对所有打开设备的进程一视同仁进行广播；聊天的“身份”和“来源”通过用户态程序在消息前加上的 `"[用户名]"` 前缀进行区分。不同聊天模式的本质差别主要体现在参与进程的数量和读写角色上，而驱动本身始终提供统一的多对多广播机制，从而满足实验中对“一对一、一对多、多对多聊天”的要求。

实验结果

在加载聊天字符设备驱动模块后，通过 `mknod /dev/chatdev c 290 0` 手动创建设备节

点，将主设备号 290、次设备号 0 的字符设备映射为 `/dev` 目录中的一个特殊文件；随后使用 `chmod 666 /dev/chatdev` 赋予所有用户对该设备的读写权限，以便多个普通用户进程可以同时通过 `/dev/chatdev` 参与聊天室通信。

首先，用户 A 加入聊天室并发送了一条消息，此时聊天室中只有一个用户。

随后，用户 B 加入聊天室，可以观察到 B 能够收到 A 发送的历史消息。随后，B 与 A 进行聊天，可以观察到双方的消息读写都正常运行。

![image-20251202110623550](C:\Users\OMEN\AppData\Roaming\Typora\typora-user-images\image-20251202110623550.png)

之后，用户 C 加入，进行多人聊天：

![image-20251202110830289](C:\Users\OMEN\AppData\Roaming\Typora\typora-user-images\image-20251202110830289.png)

三个用户都可以正常通过预设的设备文件进行聊天，符合预期的聊天室功能。

实验总结

实验中的问题与解决过程

1. **指导书示例代码与现代内核不兼容**
   刚开始照着实验指导书的示例写系统调用篡改模块，发现在 Linux 5.10 环境下发现无法直接修改 `sys_call_table`，示例中的接口和写法也已经过时。之后通过查资料，采用了“查出 `sys_call_table` 实际地址 + 修改 CR0 中 WP 写保护位”的方式，重新设计了模块结构：

- 使用硬编码的 `sys_call_table_addr`（由 `/proc/kallsyms` / `System.map` 获得）；
- 在 `clear_wp()` / `restore_wp()` 中通过内联汇编修改 CR0；
- 在 `hook_syscall()` / `unhook_syscall()` 中保存原函数指针、篡改并恢复 `sys_call_table[96]`。

经过多次加载、卸载模块和测试程序验证，最终在 5.10 内核上正确实现了 `SYS_gettimeofday` 的替换。

2. **字符设备驱动的状态管理一度设计不当**

在实现聊天室驱动时，一开始将读写指针都写成全局变量，结果导致所有用户共享同一个“读位置”：某个客户端一旦 `read()` 过，其他客户端就看不到之前的消息。经过调试后，意识到应该将“每个打开实例的状态”挂在 `struct file` 的 `private_data` 上，于是引入 `struct chat_private { unsigned long tail; }`，在 `open` 时为每个 `filp` 分配一份，并在 `read` / `poll` 中使用各自的 `tail`。改完后，各个客户端可以独立从全局环形缓冲区中按自己的进度阅读消息。

3. **环形缓冲区读写索引处理错误导致数据错乱**

在实现环形缓冲区时，最初简单用 `buf[head + i]`、`buf[tail + i]` 访问数组，没有考虑越界和回绕问题，当消息量较多时读取出现乱码和异常。随后结合 `BUF_SIZE` 为 2 的幂的特点，改用 `(index) & (BUF_SIZE - 1)` 实现快速取模，并在 `chat_read` 中增加“读者太慢”的检查逻辑：当 `head - tail > BUF_SIZE` 时，将 `tail` 强制追至 `head - BUF_SIZE`，只保留最新的 4KB 消息，从而解决了缓冲区覆盖和访问越界问题。

4. **`poll_wait` / `select` 协作机制不理解导致 `select()` 行为异常**

在用户程序中使用 `select()` 同时监听标准输入和 `/dev/chatdev` 时，一开始在驱动的 `chat_poll` 中只判断 `head != tail` 返回 `EPOLLIN`，没有调用 `poll_wait`，结果 `select()` 要么一直阻塞，要么无法被正确唤醒。查阅资料后了解到：`poll_wait(filp, &chat_dev.wq, wait)` 的作用是将当前进程挂到等待队列上，只有这样 `wake_up_interruptible(&chat_dev.wq)` 才能唤醒 `select()`。补上 `poll_wait` 后，`select()` 才能在新消息写入时被唤醒，聊天室程序可以正常“边收边发”。

实验收获

1. **理解了 Linux 动态内核模块的基本使用方法**

通过编写两个内核模块（系统调用篡改模块、字符设备驱动模块），实践了动态模块的使用方法：

- 模块源码编写（`module_init` / `module_exit`、`MODULE_LICENSE` 等）；
- 编写内核模块专用 Makefile，通过内核构建系统生成 `.ko`；
- 使用 `insmod` / `rmmod` 动态加载、卸载模块，结合 `lsmod`、`dmesg` 观察模块状态和日志。

对“内核功能可以在运行时按需装卸”的概念有了直观认识，也体会到模块代码一旦出错可能直接导致系统异常，必须更加谨慎。

2. **加深了对系统调用实现与内核保护机制的理解**

在实现 `SYS_gettimeofday` 的篡改过程中，实践了：

- 如何通过 `sys_call_table` 将系统调用号映射到函数指针；
- CR0 中 WP 写保护位对内核只读区域的保护作用；
- 通过 `pt_regs` 在 5.10 内核中获取系统调用参数。

同时也体会到，这种硬编码地址 + 修改 `sys_call_table` 的方式虽然能完成教学实验，但在真实系统中风险很高，也印证了“内核并不鼓励直接篡改系统调用表”的说法。

3. **掌握了字符设备驱动的基本框架与 file_operations 机制**

在聊天室驱动部分，完整实现了一个基础的字符设备驱动：

- 使用 `register_chrdev` 注册主设备号，提供 `open`、`release`、`read`、`write`、`poll` 等回调；
- 利用 `struct file` + `private_data` 保存每个打开实例的状态（读指针）；
- 使用 `copy_to_user` / `copy_from_user`、`get_user` / `put_user` 在内核态与用户态之间安全传递数据。

通过这个过程，我对“VFS 通过 file_operations 将用户的系统调用分发到具体驱动”的机制有了更清晰的认识。

4. **体验了基于字符设备的一对多、多对多通信设计**

聊天室驱动将字符设备抽象为一个“共享消息通道”，所有写入 `/dev/chatdev` 的消息进入同一个环形缓冲区，所有打开该设备的进程都可以从自己的 `tail` 开始依次

读取，实现了：

- 只有两个客户端时的“一对一”聊天；
- 一个发送、多方只读时的“一对多”广播；
- 多个客户端同时读写时的典型“多对多聊天室”。

这一设计让“用驱动实现进程间通信”的概念从抽象变成了可运行的程序。

5. **熟悉了等待队列与多路复用在同步中的具体用法**

实验中同时用到了：

- 内核等待队列 + `wait_event_interruptible` / `wake_up_interruptible` 实现阻塞式读取与唤醒；
- 驱动中的 `poll` 回调 + 用户态的 `select` 协同工作，实现单线程同时等待键盘输入和设备数据。

对于“如何在内核中等待事件、如何在用户态优雅地同时等待多个 IO 源”的理解更加具体，也为后续理解 epoll、网络编程等打下基础。

意见与建议

1. **建议更新实验指导书中的示例代码**

本次在参考实验指导书示例代码实现系统调用篡改时，遇到了较多“版本不兼容”的问题，例如直接修改系统调用表的方式在 5.10 内核下已经不再适用，部分接口和写法也已过时。建议在后续版本中增加基于现代内核版本的参考代码或伪代码，以及如何获取 `sys_call_table` 地址、如何处理 CR0 的写保护等方法。这样可以减少同学在环境差异上的时间消耗，把精力更多放在理解原理上。

2. **可以在实验文档中增加对关键结构与机制的简要说明**

在写驱动时，`struct file`、`private_data`、`wait_queue_head_t`、`poll_wait` 等结构和函数需要自己查内核文档和示例才能理解。建议在实验指导书中：简要介绍：file_operations、struct file、private_data 等，并解释常见内核错误码（如 `-EFAULT`、`-ERESTARTSYS`、`-EINVAL`）的典型用途。